

Module-4 **Logic and Reasoning**

Introduction to Logic and Reasoning-Propositional Logic-First Order Logic-Inference in First Order Logic-Unification, Forward Chaining, Backward Chaining, Resolution.

Introduction to Logic and Reasoning

Logic:

- The study of **valid arguments** and inferences.
- It provides a formal system for **representing knowledge**.

Reasoning:

- The process of **drawing conclusions** from given information.
- In AI, this means using a **logical system to derive new facts**.

Logic allows AI systems to make decisions, solve problems, and understand the world in a structured, predictable way.

Propositional Logic

A basic building block

A simple form of logic that deals with propositions, which are statements that can be either true or false.

Key Components:

Propositions: Simple statements like "It is raining" (P) or "The sun is shining" (Q).

Connectives: Operators used to combine propositions.

- **Conjunction ($\wedge$):** AND ($P \wedge Q$)
- **Disjunction ($\vee$):** OR ($P \vee Q$)
- **Negation ($\neg$ or $\sim$):** NOT ($\neg P$)
- **Implication ($\implies$):** IF...THEN ($P \implies Q$)
- **Logical equivalence ($\iff$):** if and only if

Limitations: Cannot represent complex relationships or properties of objects. For example, "All students are smart" cannot be expressed.

First-Order Logic

A More Expressive Language

An extension of propositional logic that allows for **quantifiers, variables, and predicates**.

Key Components:

Predicates: Represent properties or relationships
e.g., $\text{IsStudent}(x)$, $\text{Likes}(x,y)$.

Quantifiers:

- **Universal Quantifier ($\forall$):** "For all" or "For every" ($\forall x \text{IsStudent}(x) \Rightarrow \text{Smart}(x)$).
- **Existential Quantifier ($\exists$):** "There exists" or "There is at least one" ($\exists x \text{Likes}(x, \text{Apples})$).

Advantages: Can model more complex, real-world scenarios and handle relationships between objects.

Inference in First-Order Logic

Making Inferences in First-Order Logic

The process of deriving **new logical sentences** from existing ones.

Key Concepts:

Knowledge Base (KB): A set of logical sentences (axioms, facts).

Inference Rule: A method for generating **new sentences from the KB**.

Soundness and Completeness:

- **Soundness:** An inference procedure is sound **if every derived sentence is true**.
- **Completeness:** An inference procedure is complete **if it can derive every true sentence**.

Unification

- Finding the **Right Substitution**
- The process of **finding substitutions that make two logical expressions identical.**

Example:

Expression 1: Knows(John,x)

Expression 2: Knows(John,Jane)

Unification: The substitution {x/Jane} makes both expressions identical.

- It is a core algorithm used in many logical AI systems (like Prolog) and is a foundational step in more complex inference methods.

Forward Chaining

Forward Chaining: **Data-Driven Reasoning**

A reasoning method that starts with **the known facts and applies inference rules to derive new facts until a goal is reached.**

Analogy: You're a detective collecting clues and seeing where they lead you.

Process:

- Start with facts in the knowledge base.
- Find an inference rule whose premises match the facts.
- Add the conclusion of the rule to the knowledge base.
- Repeat until the goal is found.

Use Case: **Good for scenarios where all the facts are known upfront, and you want to see all possible consequences.**

Backward Chaining

Backward Chaining: **Goal-Driven Reasoning**

A reasoning method that **starts with the goal and works backward to find the facts needed to prove it.**

Analogy: You're trying to solve a puzzle, so you start with the last piece and work backward.

Process:

- Start with the goal you want to prove.
- Find an inference rule whose conclusion matches the goal.
- Try to prove the premises of that rule as new sub-goals.
- Repeat until all sub-goals are proven by facts in the knowledge base.

Use Case: **Ideal for diagnosis and planning, where you have a specific target in mind.**

Resolution

Resolution: **A Powerful Refutation System**

A complete inference rule for first-order logic that is used to prove a goal by showing **that its negation leads to a contradiction**.

To prove that A is true, you assume A is false ($\neg A$), and then you show that this assumption leads to a contradiction (a conflict, like P and $\neg P$).

Key Steps:

- Convert all sentences to a standard form called **Conjunctive Normal Form (CNF)**.
- **Negate** the goal you want to prove.
- Add the negated goal to **the set of sentences**.
- Repeatedly apply **the resolution rule to find a contradiction** (the empty clause).

The most popular inference procedure for automated theorem proving.

Key Takeaways:

Logic is the foundation of structured AI reasoning.

Propositional Logic is a basic building block, while First-Order Logic provides more expressive power.

Inference methods like Forward Chaining, Backward Chaining, and Resolution allow AI to draw conclusions from knowledge.

Unification is the key to matching patterns in logical expressions.

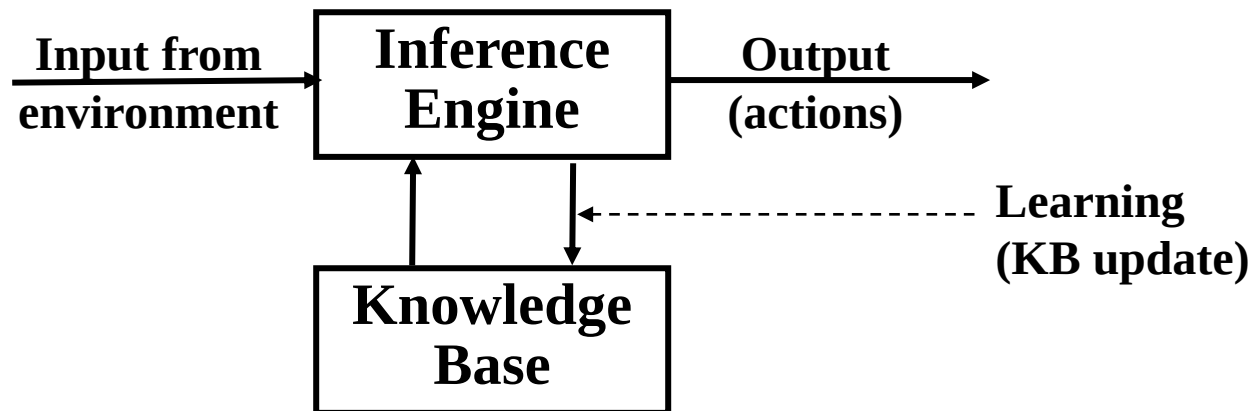
Future of Logic in AI: Continues to evolve, especially in areas like explainable AI (XAI) and knowledge graphs.

Agents that Reason Logically

A Knowledge-Based Agent

- A knowledge-based agent consists of a **knowledge base (KB)** and an **inference engine (IE)**.
- A knowledge-base is a **set of representations of what one knows about the world** (objects and classes of objects, the fact about objects, relationships among objects, etc.)
- Each individual representation is called a **sentence**.
- The sentences are expressed in a **knowledge representation language**.
- Examples of sentences
 - The moon is made of green chilies
 - If A is true then B is true
 - A is false
 - All humans are mortal
 - Confucius is a human

- The **Inference engine** derives **new sentences** from the input and KB
- The inference mechanism **depends on the representation in the KB**
- The agent operates as follows:
 1. It receives percepts from the environment
 2. It computes what action it should perform (by IE and KB)
 3. It performs the chosen action (some actions are simply inserting inferred new facts into KB).



KB can be viewed at different levels

- **Knowledge Level**

- The **most abstract level**: describe agent by saying what it knows.
- Example: A taxi agent might know that the **Golden Gate Bridge connects San Francisco with the Marin County**.

- **Logical Level**

- The level at which **the knowledge** is **encoded into sentences**.
- Example: **Links(GoldenGateBridge, SanFrancisco, MarinCounty)**.

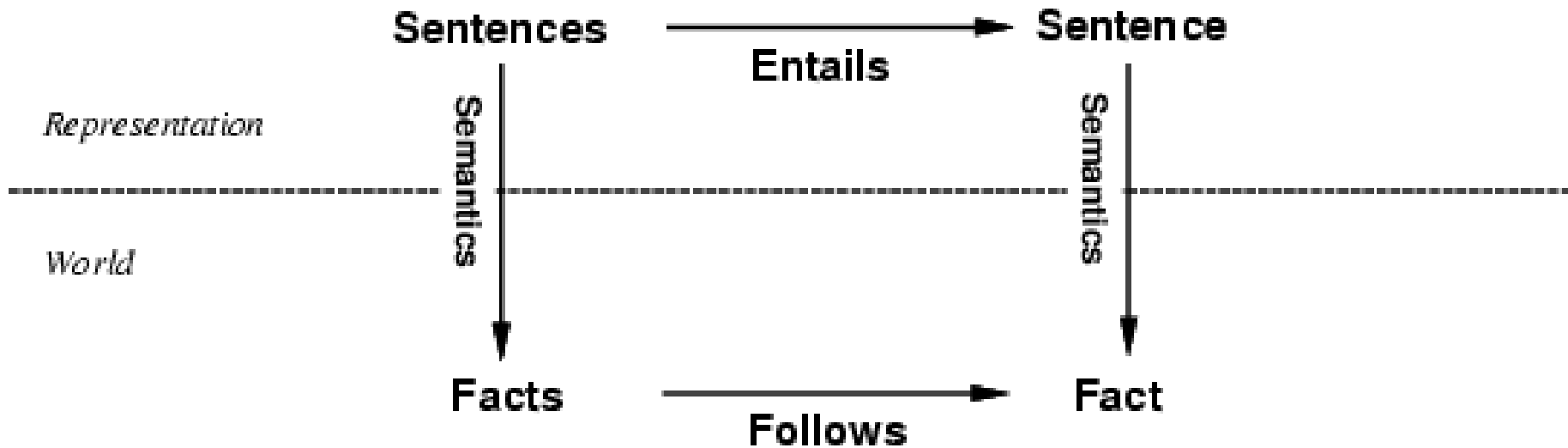
- **Implementation Level**

- The physical representation of the sentences in the logical level.
- Example: **“(Links GoldenGateBridge, SanFrancisco, MarinCounty)”**

Representation, Reasoning, and Logic

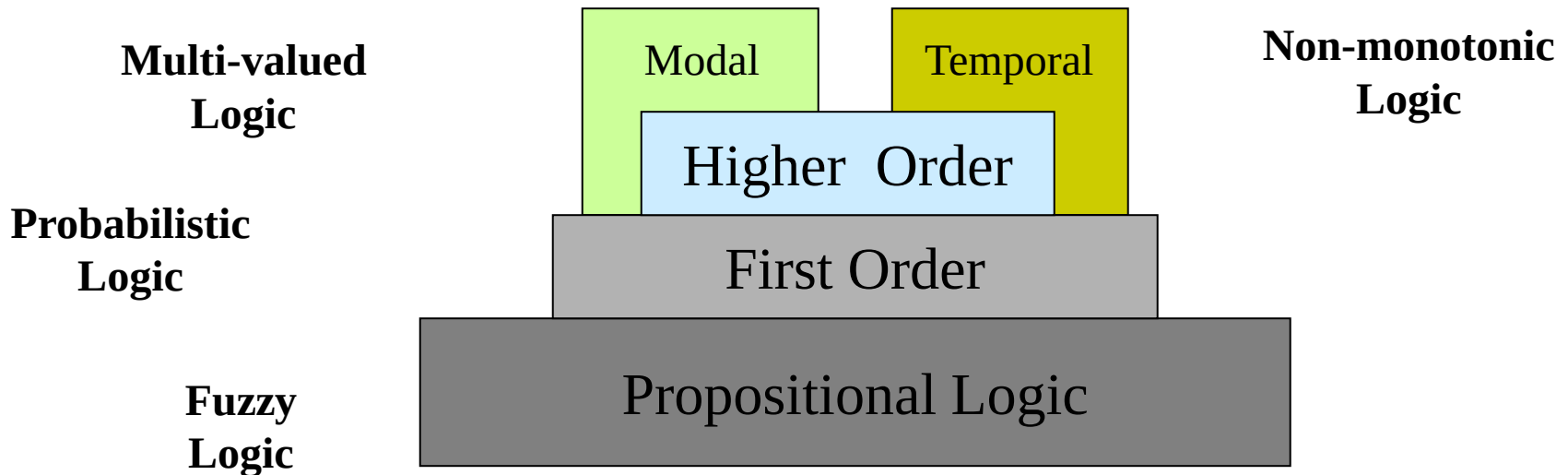
- The objective of knowledge representation is **to express knowledge in a computer-tractable form, so that agents can perform well.**
- A knowledge representation language is defined by:
 - Its **syntax**, which defines all possible sequences of symbols that constitute sentences of the language (**grammar** to form sentences)
 - Its **semantics** determines the facts in the world to which the sentences refer (**meaning** of sentences)
 - Each sentence makes a claim about the world.
 - Its **proof theory** (**inference rules and proof procedures**)

The Connection between Sentences and Facts



- **Semantics maps sentences in logic to facts in the world.**
- The property of one fact following from another is mirrored by the property of one sentence being entailed by (inferred from) another.

Logic as a KR language



Propositional Logic: Syntax

- **Symbols:**

Logical constants: true (T), false (F)

Propositional symbols: P, Q, S, ...

logical connectives:

$\wedge$...conjunction (and) $\vee$...disjunction (or)

$\sim$...negation (not) $\Rightarrow$...implication (if)

$\Leftrightarrow$...logical equivalence (if and only if)

Wrapping **parentheses**: (...)

- A **proposition** (denoted by a proposition symbol) is a declarative statement which can be either true or false but not both or neither.

– The moon is made of green chilies (F)

– VIT is closer to Chennai than to Hyderabad (T)

– $P = NP$ (truth unknown)

- **Sentence**

1. T or F itself is a sentence
2. Individual proposition symbols P, Q, ... are sentences
3. If S is a sentence, so is (S)
4. If S1 and S2 are sentences, so are
 $S1 \wedge S2$, $S1 \vee S2$, $S1 \Rightarrow S2$, $S1 \Leftrightarrow S2$, $\sim S1$
5. Nothing else is a sentence

- **Order of precedence of logical connectors**

$\sim$, $\wedge$, $\vee$, $\Rightarrow$, $\Leftrightarrow$

- Minimum set of logical connectors

$(\sim , \wedge)$, or $(\sim , \vee)$

- **Atomic sentences:** T, F, P, Q, ...

- **Literals:** atomic sentences and their negations

Knowledge Representation and Reasoning

Outline

- **Knowledge-based agents**
- **Wumpus world**
- **Logic in general - models and entailment**
- **Propositional (Boolean) logic**
- **Equivalence, validity, satisfiability**
- **Inference rules and theorem proving**
 - **resolution**
 - **forward chaining**
 - **backward chaining**

Knowledge representation

- **Knowledge:** Facts, information, and skills acquired through experience or education
- **Knowledge Representation:** Representing **information about the real world** in some formal language.
- It also enables an intelligent machine to **learn from that knowledge** and experiences so that it can behave intelligently like a human.



“Navy sedan with crimson hatches”

Semantic Matching

Navy = Blue
Crimson = Red
Sedan = Car
Hatch = Door



A Red Car with
Blue Doors



A Navy Sedan with
Crimson Hatches



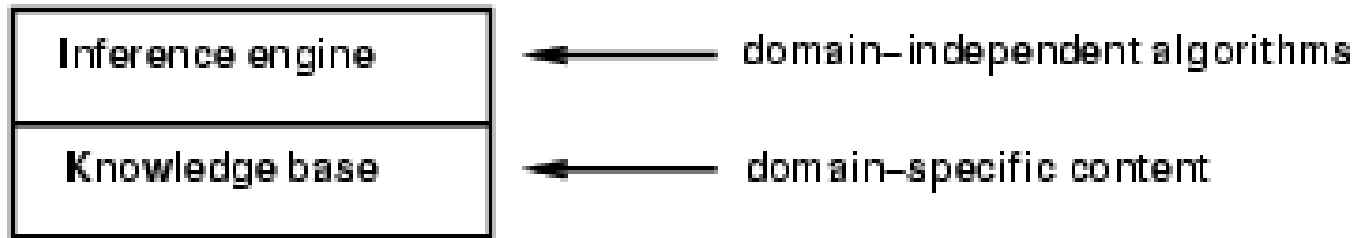
A blue Car AND a
Red Door



A car,
A Red Door,
A Blue chair

Knowledge bases

A typical **expert system** consists of a **knowledge base** and an **inference engine**. The knowledge base **stores world facts**. The inference engine **applies logical rules to the knowledge base** and deduces new knowledge.



- Knowledge base = set of **sentences** in a **formal** language
- **Declarative** approach to building an agent (or other system):
 - **Tell it what it needs to know**
- Then it can **Ask** itself **what to do next** - answers should follow from the KB
- Agents can be viewed at the **knowledge level**
i.e., what they know, regardless of how implemented
- Or at the **implementation level**
 - i.e., data structures in KB and algorithms that manipulate them.

Types of Knowledge



Types of knowledge

knowledge which needs to be represented in AI systems:

1. Declarative Knowledge:

- Declarative knowledge is to know about something.
- It includes concepts, facts, and objects.
- It is also called descriptive knowledge and **expressed in declarative sentences.**
- It is simpler than procedural language.

2. Procedural Knowledge

- It is also known as imperative knowledge.
- Procedural knowledge is a type of knowledge which is responsible for knowing **how to do something.**
- It can be directly applied to any task.
- It includes rules, strategies, procedures, agendas, etc.
- Procedural knowledge depends on the task on which it can be applied.

Types of knowledge

3. Meta-knowledge:

- This is **knowledge about knowledge** and is often used to reason about and improve the performance of AI systems.
- Describe the knowledge of pre-defined knowledge.

3. Heuristic knowledge:

- Heuristic knowledge represents **the knowledge of some experts in a field or subject.**
- Heuristic knowledge is rules of thumb based on previous experiences, awareness of approaches, and are good to work but not guaranteed.

3. Structural knowledge:

- **Structural knowledge is basic knowledge for problem-solving.**
- It describes relationships between various concepts such as kind of, part of, and grouping of something.
- It **describes the relationship that exists between concepts or objects.**

Knowledge Base Vs Database

More Rules	Less Rules
Facts Less	More Facts
Explicit Rules and Facts	Explicit Facts and Implicit Rules
Experts update	Clerks update
Main Memory Based	Disk Based

Knowledge Based Systems

- Knowledge-based systems are artificial intelligence tools working in **a narrow domain to provide intelligent decisions with justification**. Knowledge is acquired and represented using various techniques: rule-based systems, frames, scripts, propositional logic, and First-order logic.
- **Knowledge-based systems** are systems based on the methods and technique of Artificial Intelligence. Their core components are the **knowledge base and the inference mechanisms**

Knowledge Representation

- Knowledge representation and knowledge engineering are **central to AI research.**
- Many of the problems machines are expected **to solve will require extensive knowledge about the world.**
- Among the things that AI needs to represent are: *objects, properties, categories and relations between objects; situations, events, states and time; causes and effects; knowledge about knowledge (what we know about what other people know); and many other, less well researched domains.*

Issues in Knowledge Representation

- What is the **nature of knowledge**, and how do we represent it?
- Should a **representation scheme** deal with a particular domain, or **should it be general-purpose**?
- How **expressive is a representation scheme** or formal language?
- Should the scheme be **declarative or procedural**?

A simple knowledge-based agent

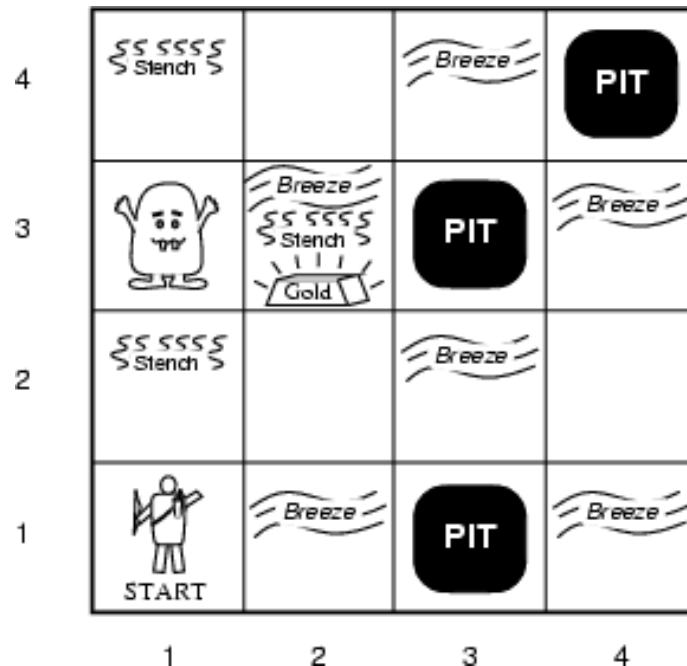
```
function KB-AGENT(percept) returns an action
  static: KB, a knowledge base
         t, a counter, initially 0, indicating time

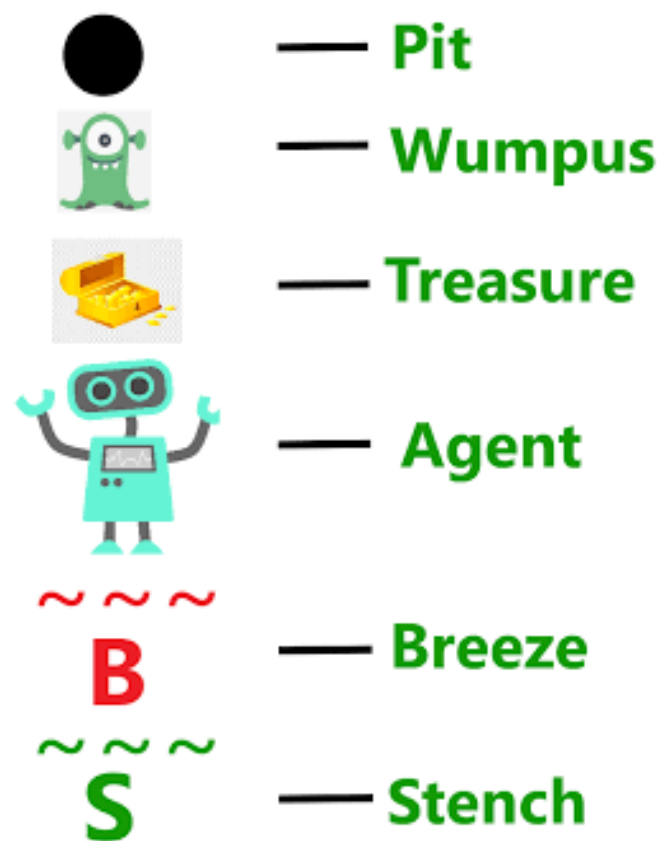
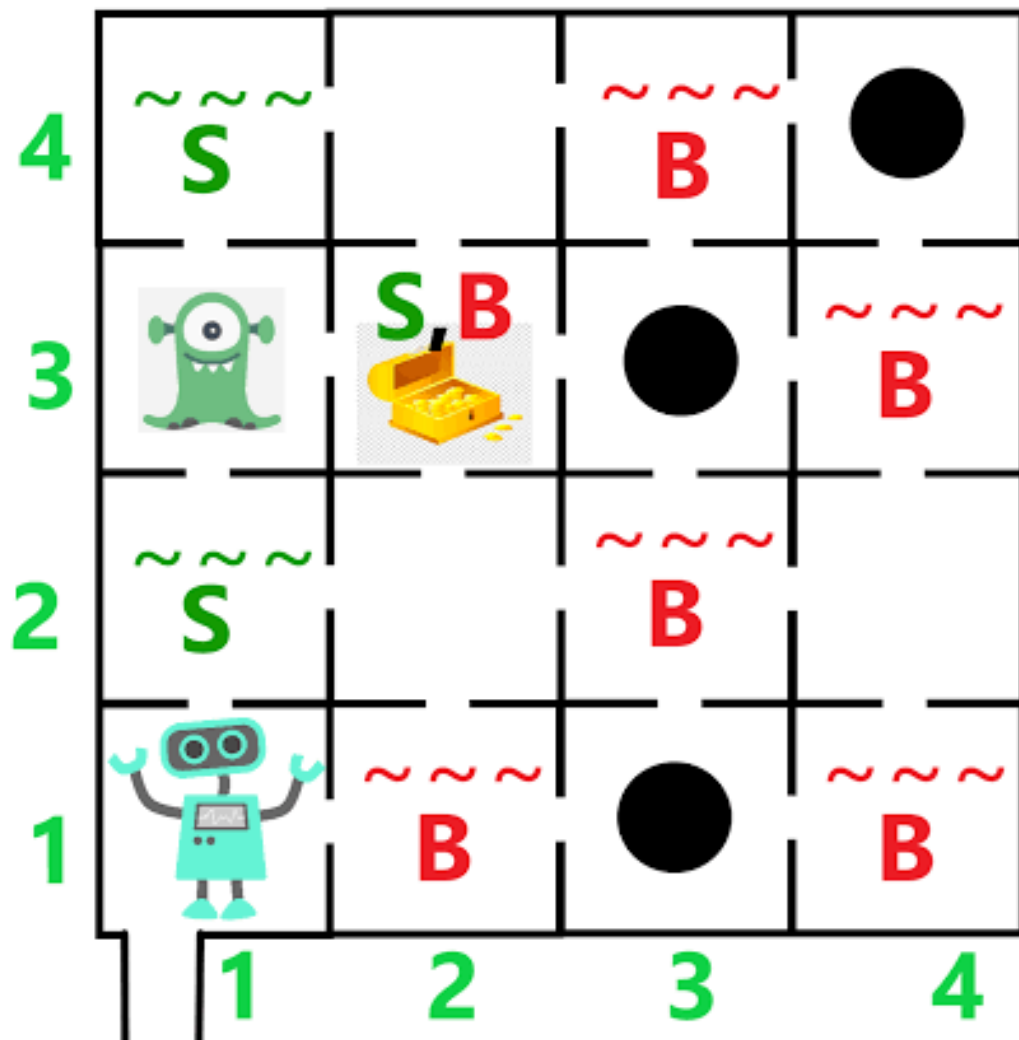
  TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))
  action ← ASK(KB, MAKE-ACTION-QUERY(t))
  TELL(KB, MAKE-ACTION-SENTENCE(action, t))
  t ← t + 1
  return action
```

- The agent must be able to:
 - Represent states, actions, etc.
 - Incorporate new percepts
 - Update internal representations of the world
 - Deduce hidden properties of the world
 - Deduce appropriate actions

Knowledge-Based Agent

- The **Wumpus world** is a simple world example to illustrate the worth of a knowledge-based agent and to represent knowledge representation. It was inspired by the video game **Hunt the Wumpus**, created by Gregory Yob in 1973.





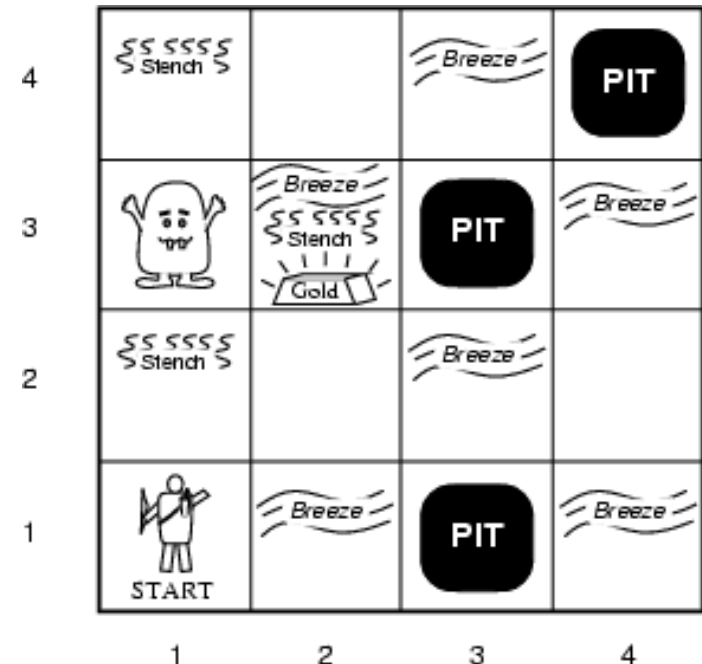
Wumpus world

Some components can also help the agent navigate the cave.
These components are given as follows:

1. The rooms adjacent to the Wumpus room are smelly, so they will have some stench.
2. The room adjacent to PITs has a breeze, so if the agent gets near a PIT, then he will perceive the breeze.
3. There will be glitter in the room if and only if the room has gold.
4. When an agent walks into a wall, it will perceive a Bump.
5. When the Wumpus is killed, it emits a woeful Scream that can be perceived anywhere in the cave.

Wumpus World PEAS description

- **Performance measure**
 - gold +1000, death -1000
 - -1 per step, -10 for using the arrow
- **Environment**
 - Squares adjacent to the Wumpus are smelly
 - Squares adjacent to a pit are breezy
 - Glitter if gold is in the same square
 - Shooting kills the wumpus if you are facing it
 - Shooting uses up the only arrow
 - Grabbing picks up gold if in the same square
 - Releasing drops the gold in the same square
- **Sensors: Stench, Breeze, Glitter, Bump, Scream**
- **Actuators:** Left turn, Right turn, Forward, Grab, Release, Shoot



Wumpus world

- Sensors: The agent has five sensors, each of which gives a single bit of information: –
- In the square containing the wumpus and in the directly (not diagonally) adjacent squares, the agent will perceive a **Stench**.
- In the squares directly adjacent to a pit, the agent will **perceive a Breeze**.
- In the square where the gold is, the agent will **perceive a Glitter**.
- When an agent walks into a wall, it will **perceive a Bump**.
- When the wumpus is killed, it **emits a woeful Scream** that can be perceived anywhere in the cave.

1,4	2,4	3,4	4,4
1,3 W!	2,3	3,3	4,3
1,2 A S OK	2,2 OK	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1

A = Agent
B = Breeze
G = Glitter, Gold
OK = Safe square
P = Pit
S = Stench
V = Visited
W = Wumpus

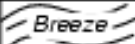
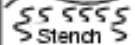
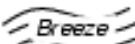
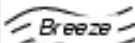
1. [None, None, None, None, None]
2. [None, Breeze, None, None, None].
3. [Stench, None, None, None, None]
4. [Stench, Breeze, Glitter, None, None].

Wumpus world characterization

- Fully_Observable No – only local perception
- Deterministic Yes – outcomes exactly specified
- Episodic No – sequential at the level of actions
- Static Yes – Wumpus and Pits do not move
- Discrete Yes. Finite number of actions, rooms, and outcomes.
-
- Single-agent? Yes – Wumpus is essentially a natural feature

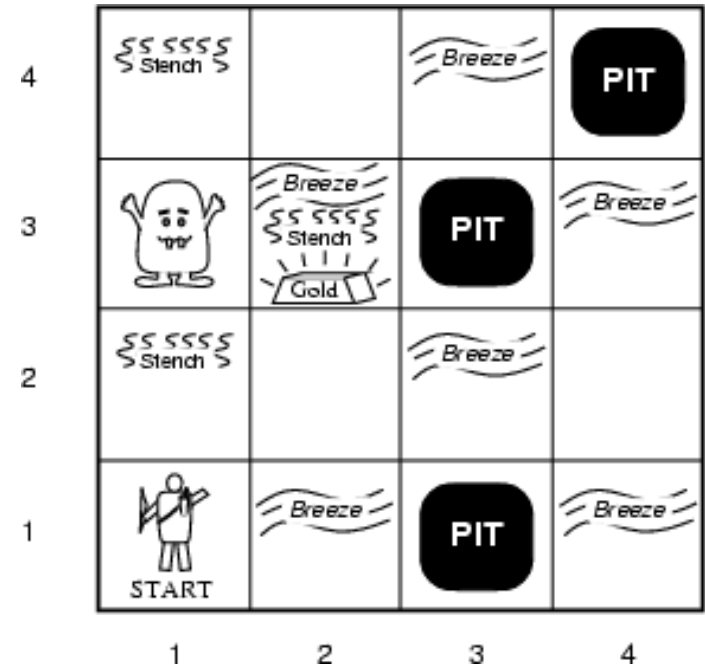
Exploring a wumpus world

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2	3,2	4,2
1,1 A OK	2,1 OK	3,1	4,1

4	 Stench		 Breeze	 PIT
3		 Breeze  Stench  Gold	 PIT	 Breeze
2	 Stench		 Breeze	
1	 START	 Breeze	 PIT	 Breeze
	1	2	3	4

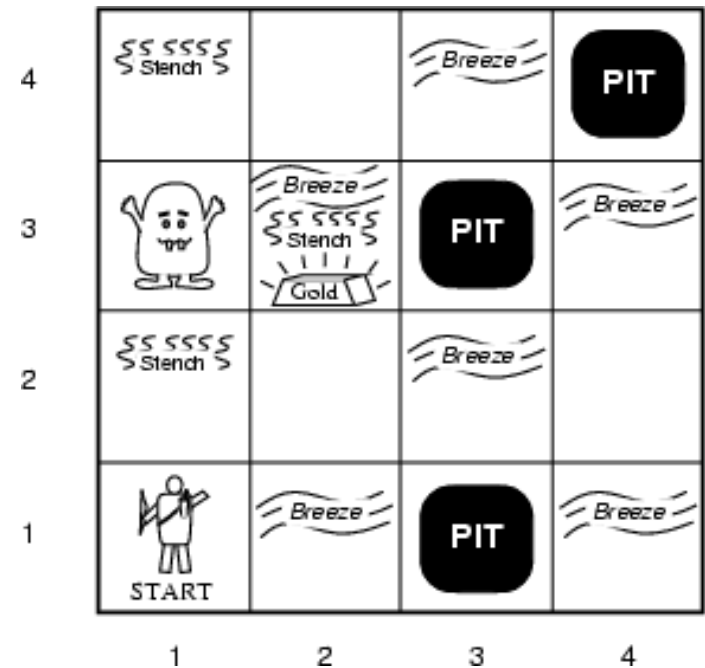
Exploring a wumpus world

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2 P?	3,2	4,2
1,1 V OK	2,1 A B OK	3,1 P?	4,1



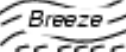
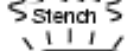
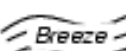
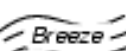
Exploring a wumpus world

1,4	2,4	3,4	4,4
1,3 W!	2,3	3,3	4,3
1,2 A S OK	2,2 OK	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1



Exploring a wumpus world

1,4	2,4 P?	3,4	4,4
1,3 W!	2,3 A S G B	3,3 P?	4,3
1,2 S V OK	2,2 V OK	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1

4	 Stench		 Breeze	PIT
3		 Breeze  Stench  Gold	PIT	 Breeze
2	 Stench		 Breeze	
1	 START	 Breeze	PIT	 Breeze
	1	2	3	4

Logic in general

- **Logics** are formal languages for representing information such that conclusions can be drawn
- **Syntax** defines the sentences in the language
- **Semantics** define the "meaning" of sentences;
 - i.e., define **truth of a sentence in a world**
- E.g., the language of arithmetic
 - $x+2 \geq y$ is a sentence; $x^2+y > \{ \}$ is not a sentence
 - $x+2 \geq y$ is true iff the number $x+2$ is no less than the number y
 - $x+2 \geq y$ is true in a world where $x = 7, y = 1$
 - $x+2 \geq y$ is false in a world where $x = 0, y = 6$

Propositional Logic:

- **Logical Connectives**
- **Truth Table**
- **Precedence of connectives**
- **converting sentences into logical form**
- **Equivalent**
- **Tautology**
- **Contradiction**
- **Logical Implication**
- **Valid/Invalid, Satisfiable/Unsatisfiable**
- **Logical Entailment**
- **Inference**
- **Modus Ponens, Modus Tollens**
- **Resolution**
- **Conjunctive Normal Form**
- **Disjunctive Normal Form**

Propositional logic: Syntax

- Propositional logic (PL) is the simplest form of logic where **all statements are made up of propositions**. A proposition is a declarative statement that is **either true or false**.
- It is a technique of knowledge representation in logical and mathematical form.
- Propositional logic is the **simplest logic that illustrates basic ideas (Boolean Logic)**
- The proposition symbols P_1, P_2 etc., are sentences
 - If S is a sentence, $\neg S$ is a sentence (**negation**)
 - If S_1 and S_2 are sentences, $S_1 \wedge S_2$ is a sentence (**conjunction/and**)
 - If S_1 and S_2 are sentences, $S_1 \vee S_2$ is a sentence (**disjunction/or**)
 - If S_1 and S_2 are sentences, $S_1 \Rightarrow S_2$ is a sentence (**implication/implies**)
 - If S_1 and S_2 are sentences, $S_1 \Leftrightarrow S_2$ is a sentence (**biconditional-if and only if**)

Propositional Logic (PL): Semantics

- Need an **interpretation** of symbols for a given set of sentences
 - Proposition symbols do not have meaning by themselves
 - **An interpretation connects proposition symbols to a statement about the world** (which may be true or false in that world)
 - An interpretation in PL can be defined as an **assignment of truth values** to all proposition symbols involved
 - There are many interpretations for a given set of sentences (2^n if they involve n distinct proposition symbols)
 - **Example:**
 - I_1: P: it is humid (T) Q: it is hot (T) $P \wedge Q$ is true**
 - I_2: P: moon is made of green cheese (F)**
 - Q: I am happy (T) $P \wedge Q$ is false**

Propositional logic: Semantics

Each model specifies true/false for each proposition symbol

E.g.	$P_{1,2}$	$P_{2,2}$	$P_{3,1}$
	false	true	false

With these symbols, 8 possible models, can be enumerated automatically.

Rules for evaluating truth with respect to a model m :

$\neg S$	is true iff	S is false
$S_1 \wedge S_2$	is true iff	S_1 is true and S_2 is true
$S_1 \vee S_2$	is true iff	S_1 is true or S_2 is true
$S_1 \Rightarrow S_2$	is true iff	S_1 is false or S_2 is true
i.e.,	is false iff	S_1 is true and S_2 is false
$S_1 \Leftrightarrow S_2$	is true iff	$S_1 \Rightarrow S_2$ is true and $S_2 \Rightarrow S_1$ is true

Simple recursive process evaluates an arbitrary sentence, e.g.,

$$\neg P_{1,2} \wedge (P_{2,2} \vee P_{3,1}) = \text{true} \wedge (\text{true} \vee \text{false}) = \text{true} \wedge \text{true} = \text{true}$$

A BNF Grammar of Sentences in Propositional Logic (PL)

BNF: Backus-Naur Form

```
S := <Sentence> ;  
<Sentence> := <AtomicSentence> | <ComplexSentence> ;  
<AtomicSentence> := "T" | "F" |  
                    "P" | "Q" | "S" ;  
<ComplexSentence> := "(" <Sentence> ")" |  
                    <Sentence> <Connective> <Sentence> |  
                    ~<Sentence> ;  
<Connective> := ^ | v | => | <=> ;  
<literal> := <AtomicSentence> | ~ <AtomicSentence>
```

BNF(Backus-Normal Form)

Formal grammar of propositional logic

$$\begin{aligned} \textit{Sentence} &\rightarrow \textit{AtomicSentence} \mid \textit{ComplexSentence} \\ \textit{AtomicSentence} &\rightarrow \textbf{True} \mid \textbf{False} \mid \textit{Symbol} \\ \textit{Symbol} &\rightarrow P \mid Q \mid \textbf{R} \mid \dots \\ \textit{ComplexSentence} &\rightarrow \neg \textit{Sentence} \\ &\mid (\textit{Sentence} \wedge \textit{Sentence}) \\ &\mid (\textit{Sentence} \vee \textit{Sentence}) \\ &\mid (\textit{Sentence} \Rightarrow \textit{Sentence}) \\ &\mid (\textit{Sentence} \Leftrightarrow \textit{Sentence}) \end{aligned}$$

Propositional logical connectives

Give a meaning to each logical connectives

A connective is a function (Boolean),

It does not depend on any particular interpretation (universal to all PL sentences)

It can best be defined by a truth table

The truth value of each sentence can then be calculated

Truth tables for logical connectives

P	Q	$\sim P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
T	T	F	T	T	T	T
T	F	F	F	T	F	F
F	T	T	F	T	T	F
F	F	T	F	F	T	T

Propositional Logic Connectives

Terms	Connective Symbol	Meaning
Conjunction	$\wedge$	AND
Disjunction	$\vee$	OR
Negation	$\sim$ $\neg$	NOT
Conditional	$\Rightarrow$	IF..THEN
Biconditional	$\Leftrightarrow$	IF AND ONLY IF

Truth Table

P	Q	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$	$\sim P$
T	T	T	T	T	T	F
T	F	F	T	F	F	F
F	T	F	T	T	F	T
F	F	F	F	T	T	T

Truth tables for connectives

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
<i>false</i>	<i>false</i>	<i>true</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>true</i>
<i>false</i>	<i>true</i>	<i>true</i>	<i>false</i>	<i>true</i>	<i>true</i>	<i>false</i>
<i>true</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>false</i>	<i>false</i>
<i>true</i>	<i>true</i>	<i>false</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>

Translation of English sentences to propositional logic

Peter can play tennis	P
Peter cannot play tennis	$\sim P$
Peter can play tennis and badminton	$P \wedge Q$
Peter can play tennis or badminton	$P \vee Q$
If Peter can play tennis then he can play badminton	$P \Rightarrow Q$
Peter can play tennis if and only if he can play badminton	$P \Leftrightarrow Q$

Examples of PL sentences

- P means "It is hot"
- Q means "It is humid"
- R means "It is raining"
- **$P \wedge Q \Rightarrow R$**
"If it is hot and humid, then it is raining"
- **$Q \Rightarrow P$**
"If it is humid, then it is hot"
- **Q**
"It is humid."

Wumpus world sentences

Let $P_{i,j}$ be true if there is a pit in $[i, j]$.

Let $B_{i,j}$ be true if there is a breeze in $[i, j]$.

$$\neg P_{1,1}$$

$$\neg B_{1,1}$$

$$B_{2,1}$$

- "Pits cause breezes in adjacent squares"

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

$$B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$$

Entailment

- Entailment means that **one thing follows from another:**

$$KB \models \alpha$$

- Knowledge base KB entails **sentence α if and only if α is true** in all worlds where KB is true
 - E.g., the KB containing “the Giants won” and “the Reds won” entails “Either the Giants won, or the Reds won”
 - E.g., $x+y = 4$ entails $4 = x+y$
 - Entailment is a relationship between sentences (i.e., **syntax**) that is based on **semantics**

Logical equivalence

- Two sentences are {logically equivalent} iff true in same models: $\alpha \equiv \beta$ iff $\alpha \models \beta$ and $\beta \models \alpha$

$$(\alpha \wedge \beta) \equiv (\beta \wedge \alpha) \quad \text{commutativity of } \wedge$$

$$(\alpha \vee \beta) \equiv (\beta \vee \alpha) \quad \text{commutativity of } \vee$$

$$((\alpha \wedge \beta) \wedge \gamma) \equiv (\alpha \wedge (\beta \wedge \gamma)) \quad \text{associativity of } \wedge$$

$$((\alpha \vee \beta) \vee \gamma) \equiv (\alpha \vee (\beta \vee \gamma)) \quad \text{associativity of } \vee$$

$$\neg(\neg\alpha) \equiv \alpha \quad \text{double-negation elimination}$$

$$(\alpha \Rightarrow \beta) \equiv (\neg\beta \Rightarrow \neg\alpha) \quad \text{contraposition}$$

$$(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta) \quad \text{implication elimination}$$

$$(\alpha \Leftrightarrow \beta) \equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) \quad \text{biconditional elimination}$$

$$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta) \quad \text{de Morgan}$$

$$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta) \quad \text{de Morgan}$$

$$(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) \quad \text{distributivity of } \wedge \text{ over } \vee$$

$$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) \quad \text{distributivity of } \vee \text{ over } \wedge$$

De Morgan - describe a logical equivalence between negated conjunctions and disjunctions

Logical Implication

Approximating causal relationships

Many logic theoreticians are not happy about the current definition for $P \Rightarrow F$

Logical Equivalence laws in PL (both sides have the same truth table)

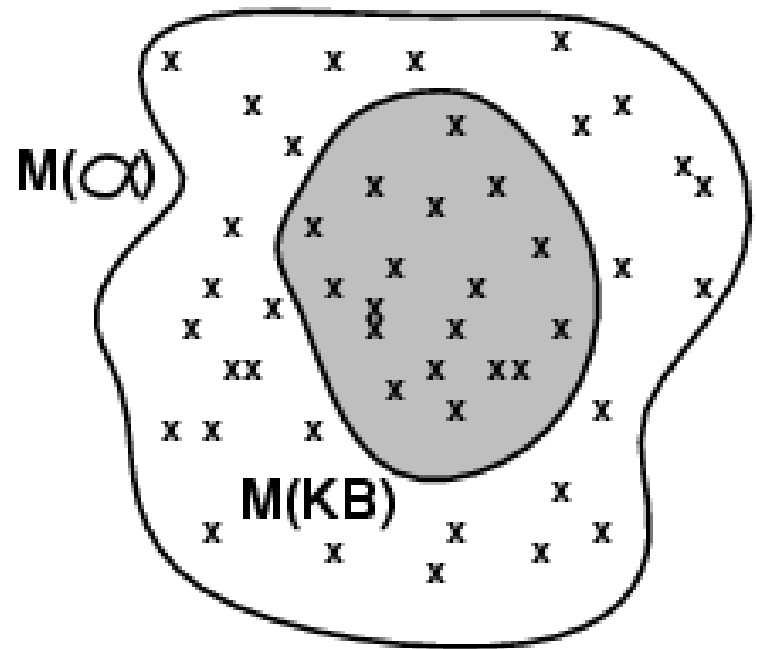
- $P \Rightarrow Q \equiv \sim P \vee Q$
- $P \Leftrightarrow Q \equiv (P \Rightarrow Q) \wedge (Q \Rightarrow P)$
- Distribution law
$$P \wedge (Q \vee R) \equiv P \wedge Q \vee P \wedge R$$
$$P \vee (Q \wedge R) \equiv (P \vee Q) \wedge (P \vee R)$$
- De Morgan's law
$$\sim(P \vee Q \vee R) \equiv \sim P \wedge \sim Q \wedge \sim R$$
$$\sim(P \wedge Q \wedge R) \equiv \sim P \vee \sim Q \vee \sim R$$

Some terms

- A **model** is an interpretation of a set of sentences such that **every sentence is True**. A model is just a formal mathematical structure that "stands in" for the world.
- A sentence is **satisfiable** if it is **True** under *some* interpretation(s)
- A **valid sentence or tautology** is a sentence that is **True under all possible interpretations**, no matter what the world is actually like or what the semantics is, e.g.,
- An **inconsistent sentence or contradiction** is a sentence that is **False under all interpretations**. The world is never like what it describes, e.g.,
- **P entails Q** (Q is a **logical consequence** of P, Q **logically follows** P), written $P \models Q$, means that whenever P is True, so is Q. In other words, all models of P are also models of Q.

Models

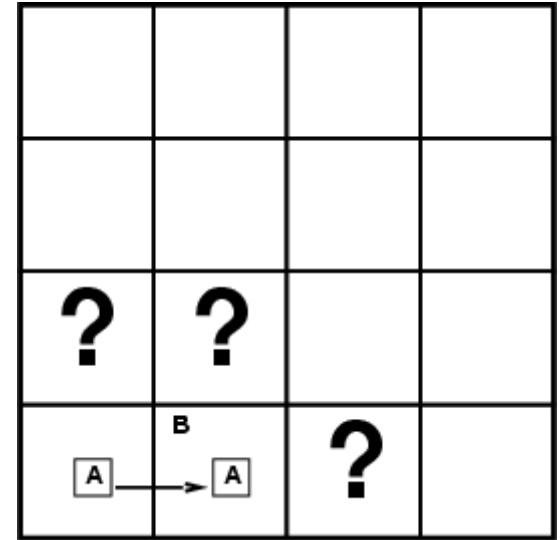
- Logicians typically think in terms of **models, which are formally structured worlds with respect to which truth can be evaluated**
- We say m is a model of a sentence α if α is true in m
- $M(\alpha)$ is the set of all models of α
- Then $KB \models \alpha$ iff $M(KB) \subseteq M(\alpha)$
 - E.g. KB = Giants won and Reds won α = Giants won



Entailment in the wumpus world

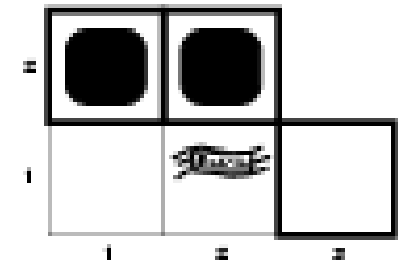
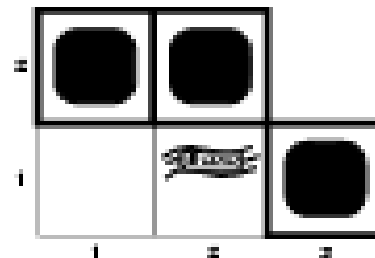
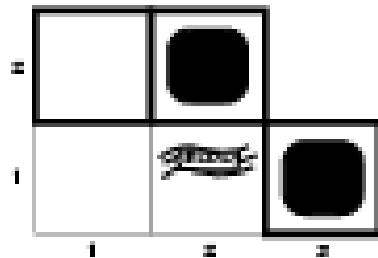
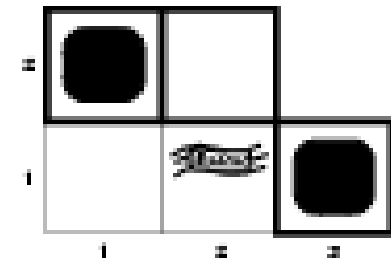
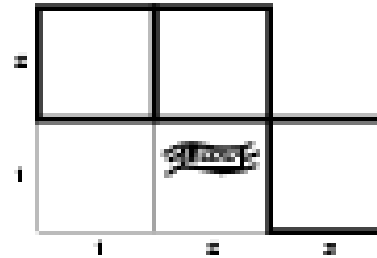
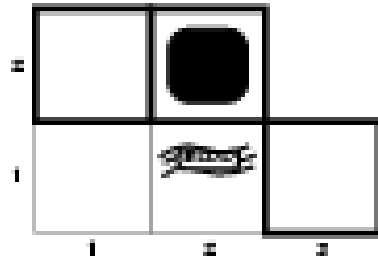
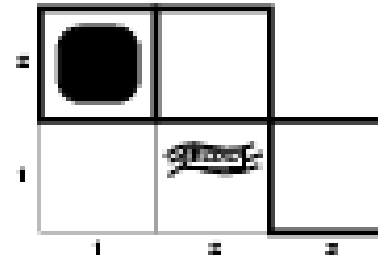
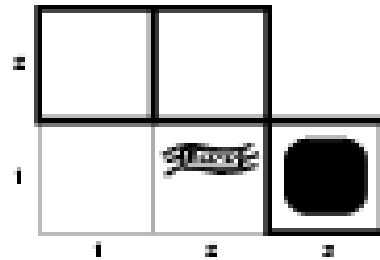
Situation after detecting nothing in
[1,1], moving right, breeze in
[1,2]

Consider possible models for
KB assuming only pits

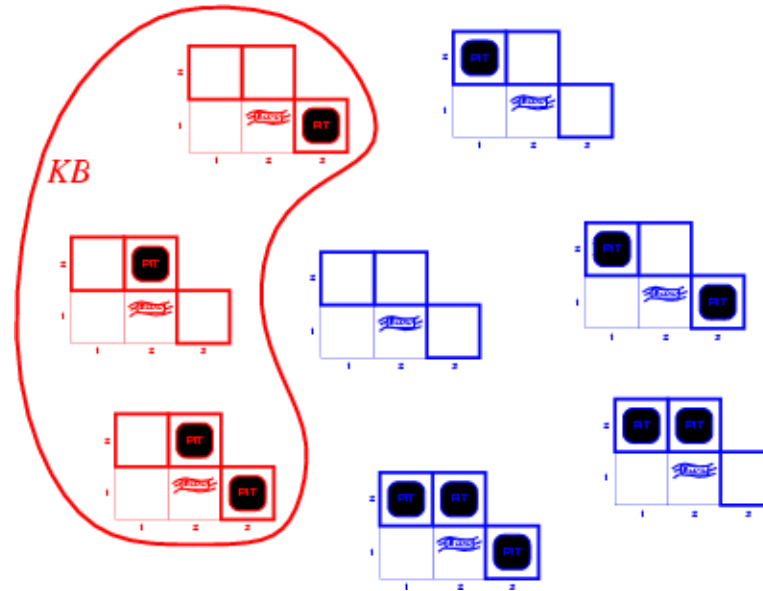


3 Boolean choices $\Rightarrow$ 8 possible
models

Wumpus models

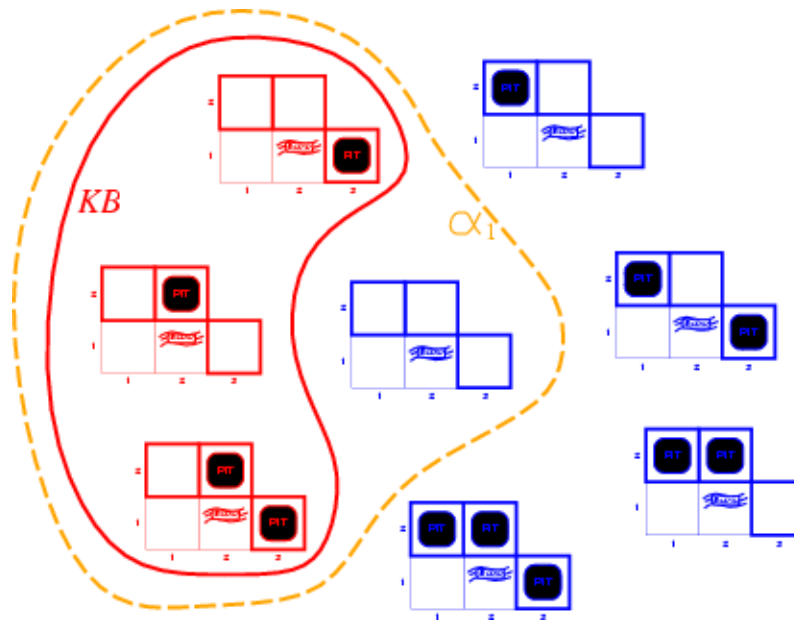


Wumpus models



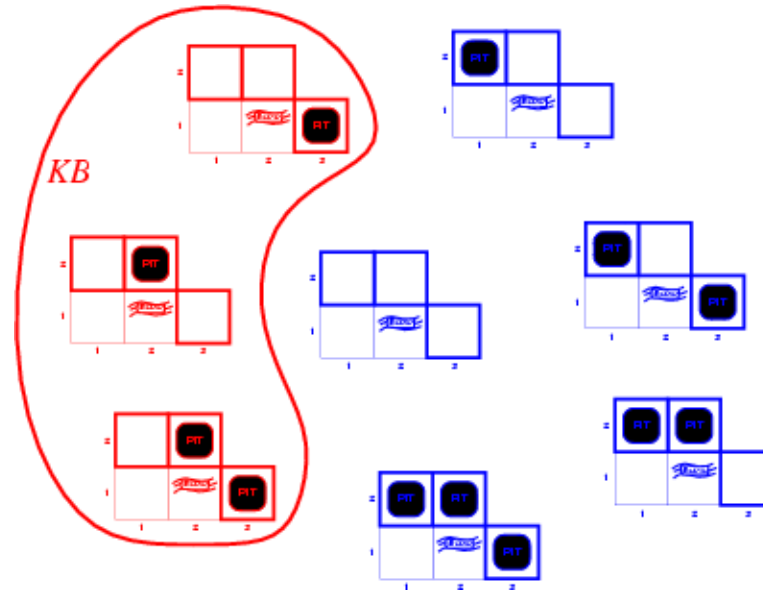
- KB = wumpus-world rules + observations

Wumpus models



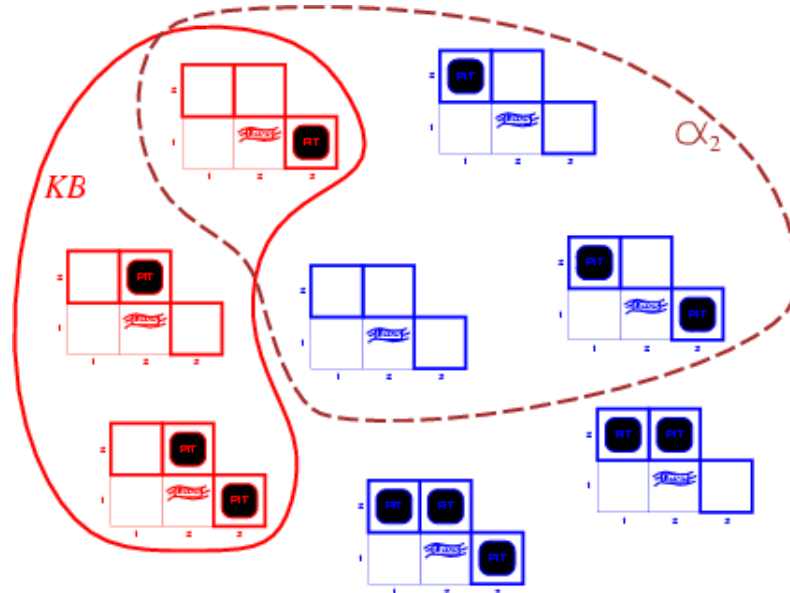
- KB = wumpus-world rules + observations
- $\alpha_1 = "[1,2] \text{ is safe}]", KB \models \alpha_1$, proved by **model checking**

Wumpus models



- KB = wumpus-world rules + observations

Wumpus models

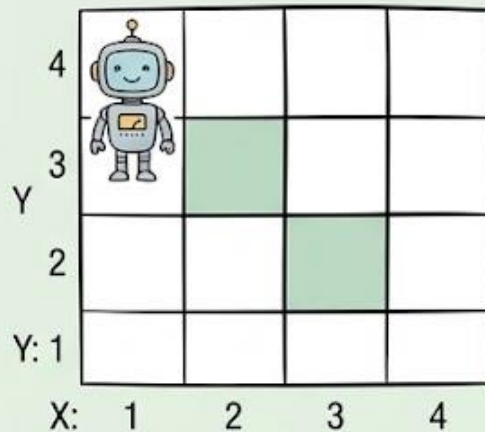


- KB = wumpus-world rules + observations
- $\alpha_2 = "[2,2] \text{ is safe}]", KB \not\models \alpha_2$
- The agent cannot conclude that $[2,2]$ is safe and must avoid stepping into it.

THE WUMPUS MODELS AND LOGICAL ENTAILMENT

KNOWLEDGE & RULES

AGENT IS SEEN AT POSITION [1,1]



[1,2], [2,2], [3,1]

PERCEPTS: [1,2]; [3,1]:

(1,1):[none],
(2,1):[breeze]

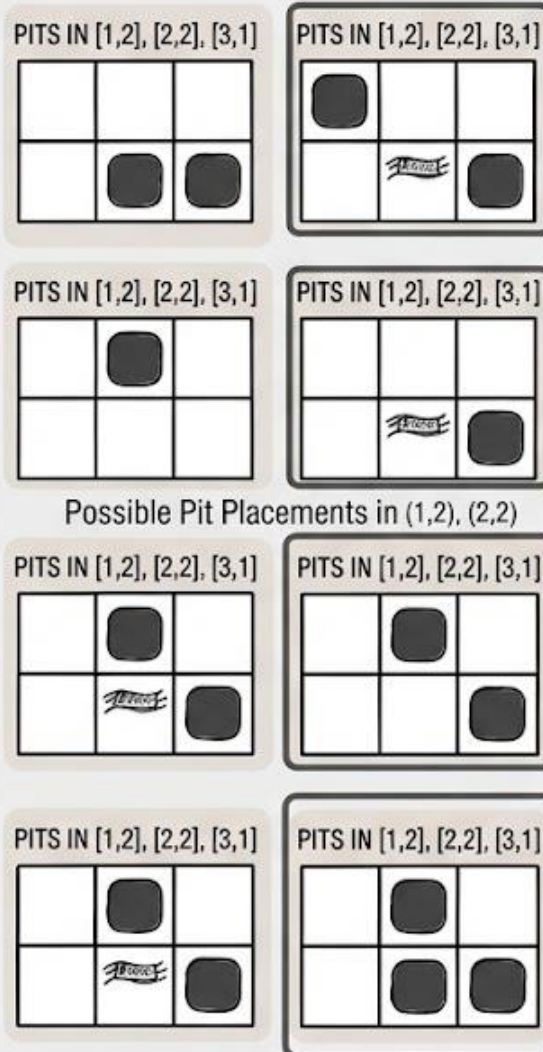
PIT DEDUCTION:

No Breeze at (1,1) -> No pit
-> at (1,2) or (2,1).

Breeze at (2,1)

-> Pit in (1,1), (2,2), or (3,1)

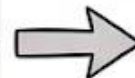
EXPLORING MODELS



Possible Pit Placements in (1,2), (2,2)

**MODELS OF KNOWLEDGE
BASE (KB)**

MODEL
CHECKING



ENTAILMENT CONCLUSION

TESTING ALPHA₁:

"Safe [1,2]" (No Pit in 1,2)

T	TRUE		
0	✓		
1	✓		
2	✓		
3	✓		✓
4	✓		✓

ALPHA₁ is true in all KB
(entailment, Safe Move)

TESTING ALPHA₂:

"Safe [2,2]" (No Pit in 2,2)

X	A	2		
1	0	T		
2	1	0		
3	1	0		
4	1	T		✗
5	F	T		✗
6	F	T	✓	✗

Alpha₂ is false KB models
(no entailment, Unsafe Move)

Validity and satisfiability

A sentence is **valid** if it is true in **all** models,

e.g., *True*, $A \vee \neg A$, $A \Rightarrow A$, $(A \wedge (A \Rightarrow B)) \Rightarrow B$

Validity is connected to inference via the **Deduction Theorem (prove that a statement to be true)**

$KB \models \alpha$ if and only if $(KB \Rightarrow \alpha)$ is valid

If a sentence is **Valid then it is tautology**

A sentence is **satisfiable** if it is true in some model

e.g., $A \vee B$, C

A sentence is **unsatisfiable** if it is true in no models

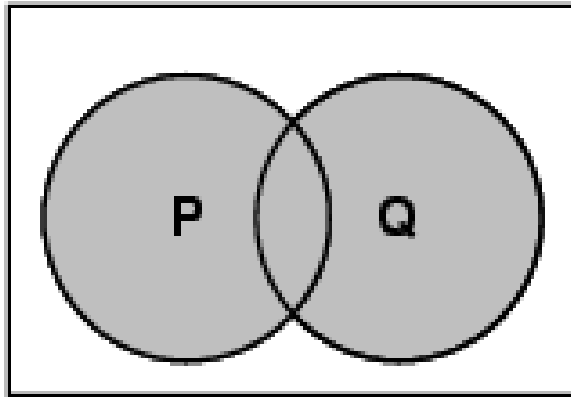
e.g., $A \wedge \neg A$

Satisfiability is connected to inference via the following:

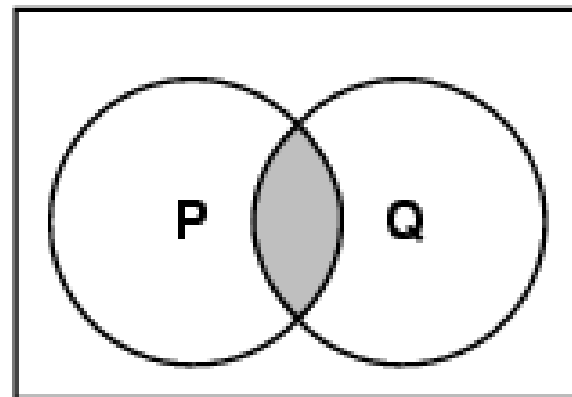
$KB \models \alpha$ if and only if $(KB \wedge \neg \alpha)$ is unsatisfiable

Models of Complex Sentences (Venn diagrams)

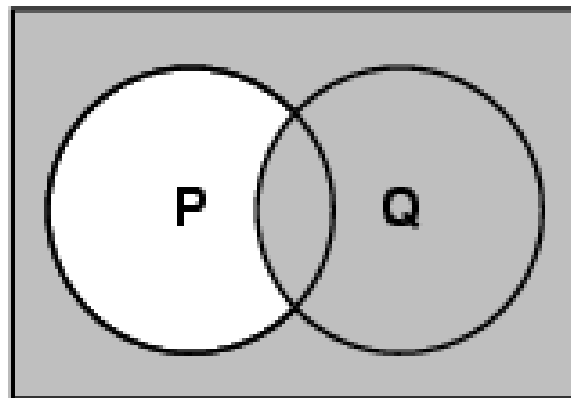
$P \vee Q$



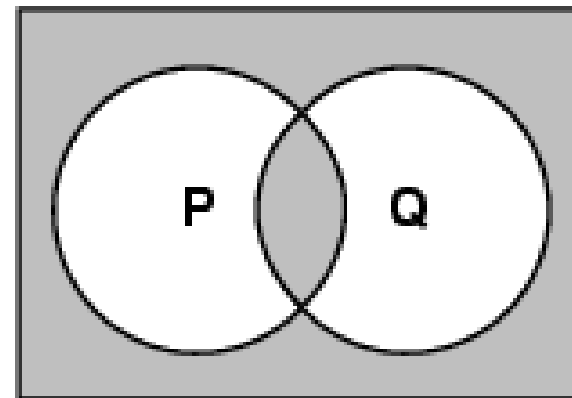
$P \wedge Q$



$P \Rightarrow Q$



$P \Leftrightarrow Q$



Inference Rule

- **Logical Inference** is **used to create new sentences** X that logically follow from a given set of sentences S (in the KB and from the input), i.e., $S \models X$
- This kind of inference is also known as **deduction**
- **Use truth table:** Whether $S \models X$ holds can be determined by whether $S \Rightarrow X$ is a tautology
 - $S \models X$ holds iff every interpretation I that makes S true also makes X true
 - $S \Rightarrow X$ is a tautology iff every interpretation I that makes S true also makes X true
 - Example: $P, P \Rightarrow Q \models Q$ because $(P, P \Rightarrow Q) \Rightarrow Q$ is true under any interpretation
 - Huge truth tables for inference involving a large number of sentences

- **Use inference rules:** generate new sentences X from one or more existing sentences S . S is called the premise, and X the conclusion of the rule.
- **Proof procedure:** a set of inference rules and a procedure for how to use these rules
- If X can be generated from S by proof procedure i , we say X is derived from S by i , denoted $S \vdash_i X$, or $S \vdash X$.
- **Soundness.** An inference procedure is sound **if every sentence** X it produces from a set of sentences S logically follows from S . (No contradiction is created).

if $S \vdash X$ then $S \models X$

- **Completeness.** An inference procedure is complete if it is able to produce every sentence that logically follows from any given S .

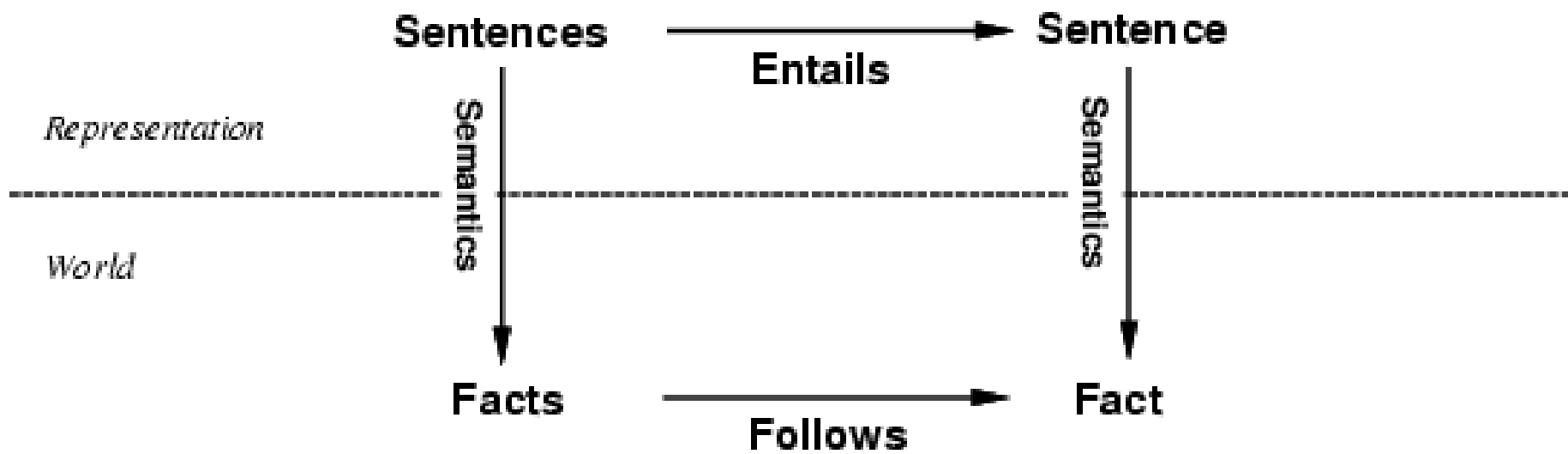
if $S \models X$ then $S \vdash X$

Soundness and Completeness:

- **Soundness:** An inference procedure is sound **if every derived sentence is true.**
- **Completeness:** An inference procedure is complete **if it can derive every true sentence.**

Inference

- $KB \vdash_i \alpha$ = sentence, α can be derived from KB by procedure I
- Soundness: i is sound if whenever $KB \vdash_i \alpha$, it is also true that $KB \models \alpha$
- Soundness means that you cannot prove anything that's wrong.
- Every derived sentence is true.
- Completeness: i is complete if whenever $KB \models \alpha$, it is also true that $KB \vdash_i \alpha$
- Completeness means that you can prove anything that's right.
- It can derive every true sentence.
- Preview: we will define a logic (first-order logic) which is expressive enough to say almost anything of interest, and for which there exists a sound and complete inference procedure.
- That is, the procedure will answer any question whose answer follows from what is known by the KB .
- The algorithm is sound because it implements the definition of entailment directly, and complete because it works for any KB .



Rules for Inference for Propositional Logic

- ◇ **Modus Ponens or Implication-Elimination:** (From an implication and the premise of the implication, you can infer the conclusion.)

$$\frac{a \Rightarrow \beta, \quad a}{\beta}$$

- ◇ **And-Elimination:** (From a conjunction, you can infer any of the conjuncts.)

$$\frac{\alpha_1 \text{ A } \alpha_2 \text{ A } \dots \text{ A } \alpha_n}{\alpha_i}$$

- ◇ **And-Introduction:** (From a list of sentences, you can infer their conjunction.)

$$\frac{\alpha_1, \alpha_2, \dots, \alpha_n}{\alpha_1 \text{ A } \alpha_2 \text{ A } \dots \text{ A } \alpha_n}$$

- 0 **Or-Introduction:** (From a sentence, you can infer its disjunction with anything else at all.)

$$\frac{\alpha_i}{\alpha_1 \vee \alpha_2 \vee \dots \vee \alpha_n}$$

- ◇ **Double-Negation Elimination:** (From a doubly negated sentence, you can infer a positive sentence.)

$$\frac{\neg \neg a}{a}$$

- ◇ **Unit Resolution:** (From a disjunction, if one of the disjuncts is false, then you can infer the other one is true.)

$$\frac{a \vee \beta, \quad \neg \beta}{a}$$

- ◇ **Resolution:** (This is the most difficult. Because 0 cannot be both true and false, one of the other disjuncts must be true in one of the premises. Or equivalently, implication is transitive.)

$$\frac{a \vee \beta, \quad \neg \beta \vee \gamma}{a \vee \gamma}$$

or equivalently

$$\frac{\neg \alpha \Rightarrow \beta, \quad \beta \Rightarrow \gamma}{\neg \alpha \Rightarrow \gamma}$$

Sound Inference Rules (deductive rules)

- Here are some examples of sound rules of inference.
- Each can be shown to be sound using a truth table -- **a rule is sound if its conclusion is true whenever the premises are true.**

<u>RULE</u>	<u>PREMISE</u>	<u>CONCLUSION</u>
Modus Ponens	$A, A \Rightarrow B$	B
Modus Tollens	$\sim B, A \Rightarrow B$	$\sim A$
And Introduction	A, B	$A \wedge B$
And Elimination	$A \wedge B$	A
Or Introduction	A	$A \vee B$
Double Negation	$\sim\sim A$	A
Chaining	$A \Rightarrow B, B \Rightarrow C$	$A \Rightarrow C$

Modus Ponens & Modus Tollens in AI

The Logic of AI Reasoning

What are Rules of Inference?

- Rules of inference are **fundamental principles of logic used to derive a conclusion from a set of premises.**
- They ensure that **if the premises are true, the conclusion must also be true.**
- In AI, these rules form the **basis for logical reasoning, enabling systems to make decisions and draw conclusions from known information.**
- Two of the most important rules are **Modus Ponens and Modus Tollens.**

Modus Ponens: The Rule of Affirming

- **Modus Ponens** (Latin for "method of affirming") is a valid, simple argument form.
- It states that **if a conditional statement is accepted** and its antecedent (the "if" part) is true, then the consequent (the "then" part) can be inferred.

If...then

- **Symbolic Form:**

- Premise 1: $P \rightarrow Q$ (If P is true, then Q is true)
- Premise 2: P is true.
- Conclusion: Q is true.

Modus Ponens in AI

- **Application:** Used in rule-based systems and expert systems to derive new facts.

- **Example: Medical Diagnosis AI**

- **Rule:** If a patient has a high fever and cough, then they may have influenza.
 - (P: Patient has high fever and cough)
 - (Q: Patient may have influenza)
- **Fact:** The patient has a high fever and cough.
 - (P is true)
- **AI Conclusion (using Modus Ponens):** The patient may have influenza.
 - (Q is true)

Modus Tollens: The Rule of Denying

- **Modus Tollens** (Latin for "method of denying") is another valid argument form.
- It states that if a conditional statement is accepted, and the consequent (the "then" part) is false, then the antecedent (the "if" part) must also be false.

Then...If

- **Symbolic Form:**

- Premise 1: $P \rightarrow Q$ (If P is true, then Q is true)
- Premise 2: $\neg Q$ is true. (Q is not true)
- Conclusion: $\neg P$ is true. (P is not true)

Modus Tollens in AI

• **Application:** Used for debugging, error analysis, and constraint satisfaction. It helps AI systems eliminate possibilities.

• **Example: Autonomous Vehicle AI**

- **Rule:** If the road is clear, then the vehicle's speed will be above 20 mph.
 - (P: The road is clear)
 - (Q: The vehicle's speed is above 20 mph)
- **Fact:** The vehicle's speed is not above 20 mph.
 - ($\neg Q$ is true)
- **AI Conclusion (using Modus Tollens):** The road is not clear.
 - ($\neg P$ is true)
- This conclusion allows the AI to trigger a new behavior, such as activating obstacle detection sensors.

Modus Ponens vs. Modus Tollens

Feature	Modus Ponens	Modus Tollens
Logic	Affirming the antecedent	Denying the consequent
Premise	P is true	$\neg Q$ is true
Conclusion	Q is true	$\neg P$ is true
AI Use Case	Forward Chaining (deriving new facts)	Backward Chaining (eliminating possibilities)

Resolution Method

- The resolution method is an **inference rule** which is used in both Propositional and First-order Predicate Logic in different ways.
- This method is used **for proving the satisfiability of a sentence**. We use the **Proof by Refutation** technique to prove the given statement in the resolution method.

Resolution Method

- The resolution method is a **complete and sound algorithm for automated theorem-proving**.
- The key idea for the resolution method is **to use the knowledge base and negated goal to obtain a null clause** (which indicates a **contradiction**).
- The resolution method is also called **Proof by Refutation**. **Since the knowledge base itself is consistent, the contradiction must be introduced by a negated goal**. As a result, we have to conclude that the original goal is true.

Resolution

The resolution method applies only to clauses that are disjunctions of literals.

Conjunctive Normal Form (CNF)

- **Conjunction of disjunctions of literal clauses**
- Logic requires that the statements be converted into Conjunctive Normal Form (CNF), **where the knowledge base is a conjunction (AND) of clauses.**
- Each **clause is itself a disjunction (OR) of literals, and the resolution** rule is then applied to these clauses to derive new clauses, ultimately aiming to **derive a contradiction** (the empty clause) to prove a goal.

Resolution is sound and complete for propositional logic

- **Soundness means that you cannot prove anything that's wrong.**
- **Completeness means that you can prove anything that's right.**

Normal forms of PL sentences

- Disjunctive normal form (DNF)
 - Any sentence can be written as a disjunction of conjunctions of literals.
 - Examples: $\underline{P \wedge Q \wedge \sim R}$; $\underline{A \wedge B} \vee \underline{C \wedge D} \vee \underline{P \wedge Q \wedge R}$; $\underline{P}$
 - Widely used in logical circuit design (simplification)
- Conjunctive normal form (CNF)
 - Any sentence can be written as a conjunction of disjunctions of literals.
 - Examples: $\underline{P \vee Q \vee \sim R}$; $(\underline{A \vee B}) \wedge (\underline{C \vee D}) \wedge (\underline{P \vee Q \vee R})$; $\underline{P}$
- Normal forms can be obtained by applying equivalence laws

$$[(A \vee B) \Rightarrow (C \vee D)] \Rightarrow P$$

$$\equiv \sim[\sim(A \vee B) \vee (C \vee D)] \vee P$$

$$\equiv [\sim\sim(A \vee B) \wedge \sim(C \vee D)] \vee P$$

$$\equiv [(A \vee B) \wedge (\sim C \wedge \sim D)] \vee P$$

$$\equiv (A \vee B \vee P) \wedge (\sim C \wedge \sim D \vee P)$$

$$\equiv (A \vee B \vee P) \wedge (\sim C \vee P) \wedge (\sim D \vee P) \quad \text{a CNF}$$

Horn Sentences

- In propositional logic, Horn sentences are a special type of logical formula that has significant importance in areas like **automated theorem proving and logic programming**.
- The key characteristic that defines them is their structure in Conjunctive Normal Form
- It has **exactly one** positive literal.
- **A logical disjunction of literals containing at most one positive (unnegated) literal.**

Disjunctive form: $\neg p \vee \neg q \vee A$

Implication form: $(p \wedge q) \rightarrow A$

Meaning: "If p and q are true, then A is true"

Conversion to CNF

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

1. Eliminate $\Leftrightarrow$, replacing $\alpha \Leftrightarrow \beta$ with $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$.
 $(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$

2. Eliminate $\Rightarrow$, replacing $\alpha \Rightarrow \beta$ with $\neg\alpha \vee \beta$.

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1})$$

3. Move $\neg$ inwards using **de Morgan's rules and double-negation**:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1})$$

4. Apply the distributivity law ($\wedge$ over $\vee$) and flatten:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$$

Resolution using Propositional Logic

Resolution Method in Propositional Logic

Consider the following Knowledge Base:

1. The humidity is high or the sky is cloudy.
2. If the sky is cloudy, then it will rain.
3. If the humidity is high, then it is hot.
4. It is not hot.

• **Goal:** It will rain.

Proving things

- A **proof** is a sequence of sentences, where **each sentence is either a premise (also called an axiom) or a sentence derived from earlier sentences** in the proof by one of the rules of inference.
- The last sentence is the **theorem** (also called goal or query) that we want to prove.
- Complete proof procedure for PL has time complexity exponential to the number of premises
- Example for the "weather problem" given before.

1 Q	Premise	"It is humid"
2 $Q \Rightarrow P$	Premise	"if it is humid, it is hot"
3 P	Modus Ponens(1,2)	"It is hot"
4 $(P \wedge Q) \Rightarrow R$	Premise	"If it's hot & humid, it's raining"
5 $P \wedge Q$	And Introduction(1)	"It is hot and humid"
6 R	Modus Ponens(4,5)	"It is raining"

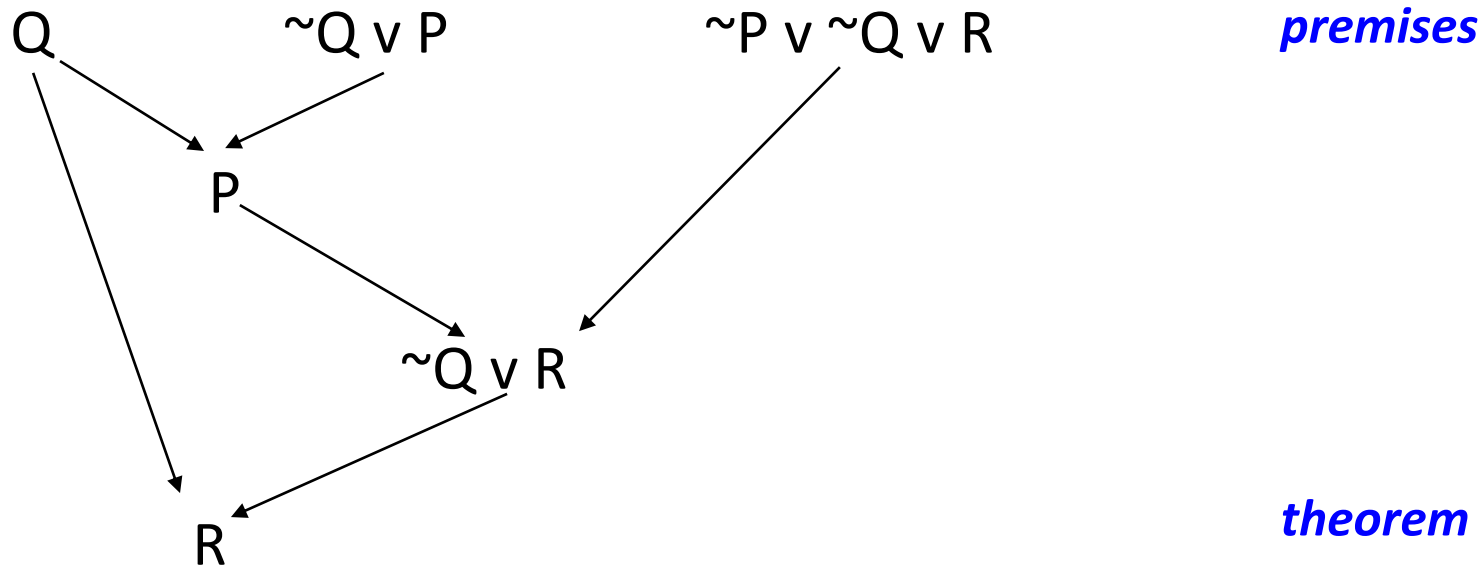
Example of Propositional Resolution

1. The humidity is high (P), the sky is cloudy (**Q**)
 2. If the sky is cloudy (Q), it will rain (R)
 3. If the humidity is high (P), it is hot. (S)
 4. It is not hot. ($\neg$ S)
- **Goal:** It will rain.

Example of Propositional Resolution

1. The humidity is high or the sky is cloudy. $P \vee Q$.
 2. If the sky is cloudy, then it will rain. $Q \rightarrow R$
 3. If the humidity is high, then it is hot. $P \rightarrow S$
 4. It is not hot. $\neg S$
- **Goal:** It will rain. $\neg R$ (Negation of Goal)

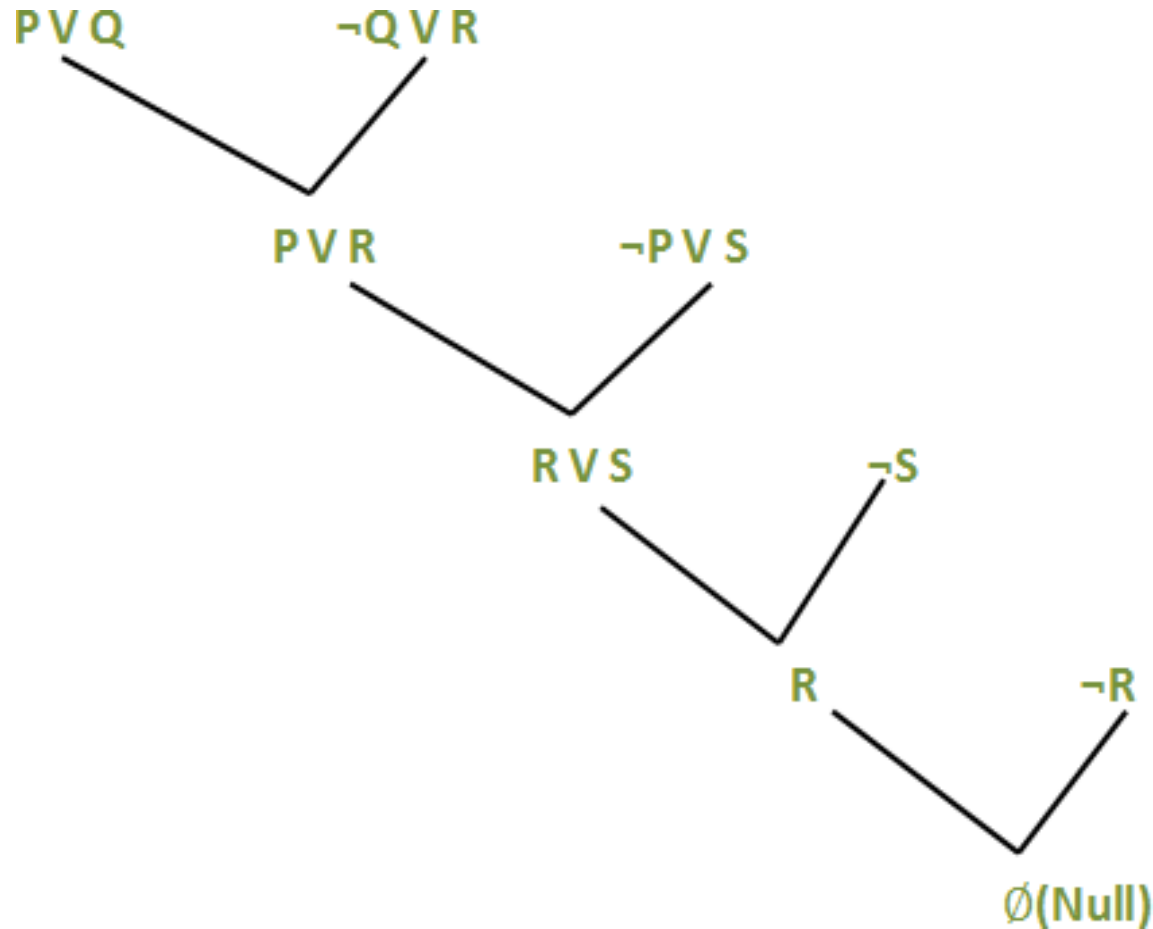
- Proof by resolution



- Theorem proving as search

- Start node: the set of given premises/axioms (KB + Input)
- Operator: inference rule (add a new sentence into parent node)
- Goal: a state that contains the theorem asked to prove
- Solution: a path from start node to a goal

Proof by Refutation (Contradiction)



After applying Proof by Refutation (Contradiction) on the goal, the problem is solved, and it has terminated with a Null clause ($\emptyset$). Hence, the goal is achieved. Thus, It is raining.

Proof with Resolution (Without Contradiction)

- Given the following hypotheses:
- If it rains, Joe brings his umbrella ($r \rightarrow u$)
- If Joe has an umbrella, he doesn't get wet ($u \rightarrow \neg w$)
- If it doesn't rain, Joe doesn't get wet ($\neg r \rightarrow \neg w$)

prove that Joes doesn't get wet ($\neg w$)

- We first put each hypothesis in CNF:
- $r \rightarrow u \equiv (\neg r \vee u)$
- $u \rightarrow \neg w \equiv (\neg u \vee \neg w)$
- $\neg r \rightarrow \neg w \equiv (r \vee \neg w)$

- We then use resolution on the hypotheses to derive the conclusion ($\neg w$):

1. $\neg r \vee u$

Premise

2. $\neg u \vee \neg w$

Premise

3. $r \vee \neg w$

Premise

4. $\neg r \vee \neg w$

L1, L2, resolution

5. $\neg w \vee \neg w$

L3, L4, resolution

6. $\neg w$

L5, idempotence

7. END

- **We can redo the previous proof (about Joe and his umbrella) using proof by contradiction with resolution:**

1. $\neg r \vee u$ Premise

2. $\neg u \vee \neg w$ Premise

3. $r \vee \neg w$ Premise

4. w Negation of conclusion

5. $\neg r \vee \neg w$ L1, L2, resolution

6. $\neg w \vee \neg w$ L3, L5, resolution

7. $\neg w$ L6, idempotence

8. NULL (FALSE) L4, L7, resolution

Steps for Resolution by Refutation

- **Negate the Goal**
- **Convert to CNF**
- **Add to Knowledge Base**
- **Apply the Resolution Rule**
- **Derive the Empty Clause**
- **Conclusion**

- **Forward Chaining**
- **Backward Chaining**

Forward chaining is a data-driven or bottom-up reasoning method.

- It starts with a set of known facts and applies inference rules to **derive new facts, continuing this process until a desired goal is reached or no more new facts can be inferred.**

Backward chaining is a goal-driven or top-down reasoning method.

- It starts with a specific goal or hypothesis and works backward to **find the facts and rules needed to prove that goal.**

Forward and backward chaining

- Inference by resolution is complete, but sometimes an overkill.
- Inference with Horn clauses can be done through forward chaining and backward chaining
- These are more efficient than the resolution algorithm.
- The resolution algorithm has exponential running time in worst case.

Horn Form

- ▶ *KB* conjunction of Horn clauses
- ▶ **Horn Clause** (at most one literal is Positive⁷)
- ▶ For example: $(\neg P \vee \neg Q \vee V)$ is a Horn Clause.
- ▶ so is $(\neg P \vee \neg W)$. But, $(\neg P \vee Q \vee V)$ is not.
- ▶ **Definite Clauses**: exactly one literal is positive.
- ▶ Horn clauses can be re-written as implications
 - ▶ proposition symbol (fact) or
 - ▶ conjunction of symbols (body or premise) $\Rightarrow$ symbol (head)
 - ▶ Example: $(\neg C \vee \neg B \vee A)$ becomes $(C \wedge B \Rightarrow A)$

Modus ponens for Horn KB:

$$\frac{\alpha_1 \dots \alpha_n, \alpha_1 \wedge \dots \alpha_n \Rightarrow \beta}{\beta}$$

Forward and backward chaining

- **Horn Form** (restricted) KB = **conjunction** of **Horn clauses**
- Horn clause = at most one literals is positive
 - » proposition symbol; or
 - » (conjunction of symbols) $\Rightarrow$ symbol
- E.g., $C \wedge (B \Rightarrow A) \wedge (C \wedge D \Rightarrow B)$

Modus Ponens: **complete** form Horn KBs

$$\frac{\alpha_1, \dots, \alpha_n \quad \alpha_1 \wedge \dots \wedge \alpha_n \Rightarrow \beta}{\beta}$$

Known as forward chaining inference rule; repeated applications until sentence of interest obtained – forward chaining algorithm.

Modus Tollens: a form of Modus Ponens

$$\frac{\neg\beta, \quad \alpha_1 \wedge \dots \wedge \alpha_n \Rightarrow \beta}{\neg(\alpha_1 \wedge \dots \wedge \alpha_n)}$$

Known as backward chaining inference rule, repeated applications until all premises obtained – backward chaining algorithm.

Both algorithms run in **linear** time

Forward chaining

- Idea: fire any rule whose premises are satisfied in the *KB*,
 - add its conclusion to the *KB*, until query is found

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

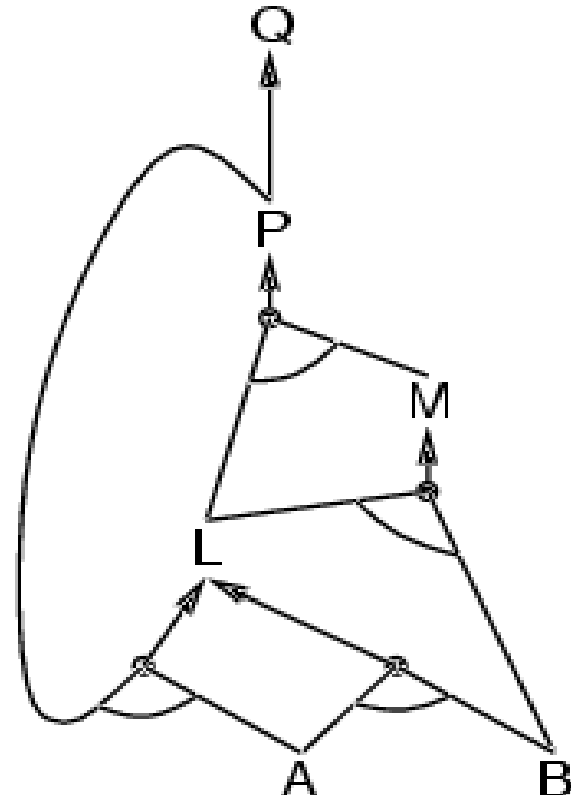
$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B



AND – OR Graph

Forward chaining algorithm

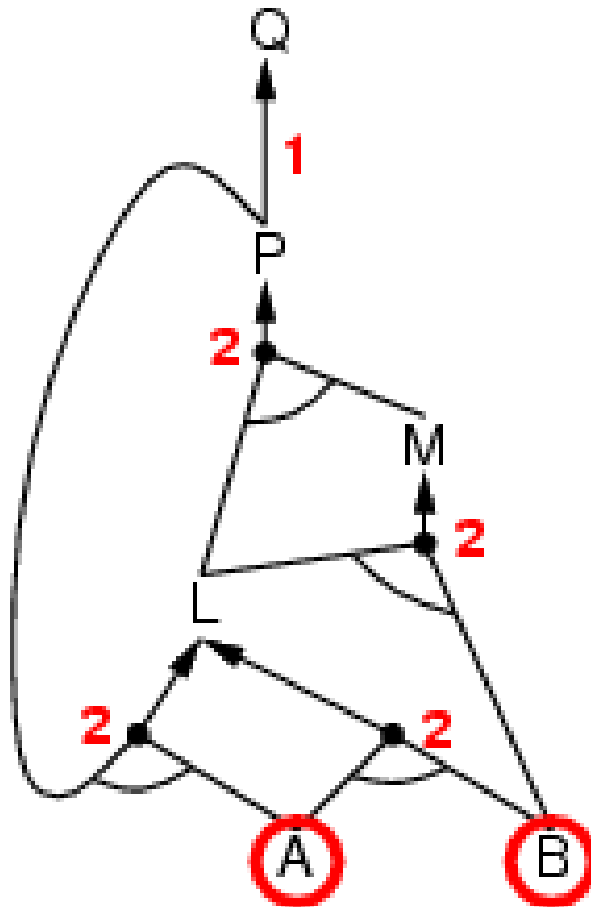
```
function PL-FC-ENTAILS?(KB, q) returns true or false
  local variables: count, a table, indexed by clause, initially the number of premises
                  inferred, a table, indexed by symbol, each entry initially false
                  agenda, a list of symbols, initially the symbols known to be true

  while agenda is not empty do
    p ← POP(agenda)
    unless inferred[p] do
      inferred[p] ← true
      for each Horn clause c in whose premise p appears do
        decrement count[c]
        if count[c] = 0 then do
          if HEAD[c] = q then return true
          PUSH(HEAD[c], agenda)

  return false
```

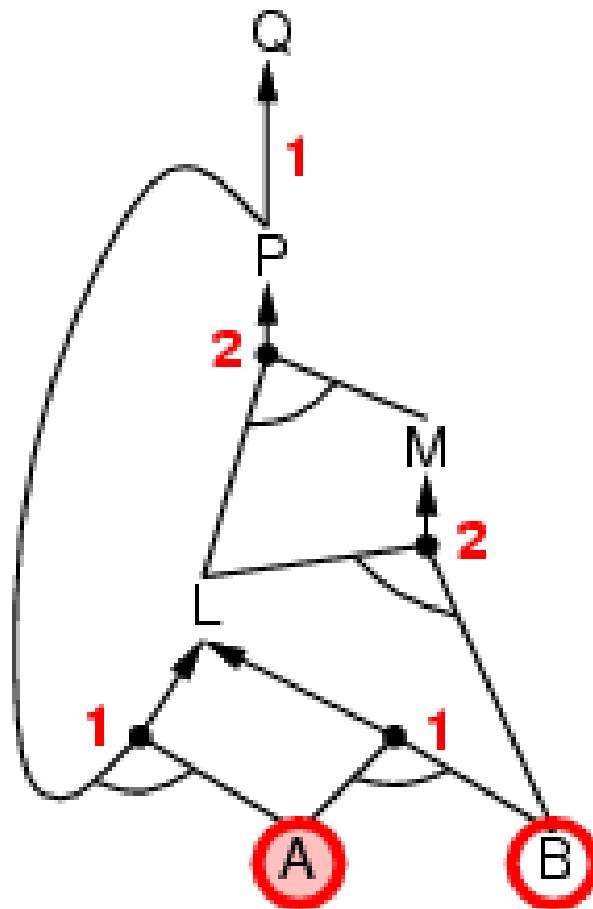
- Forward chaining is sound and complete for Horn KB

Forward chaining example



$P \Rightarrow Q$
 $L \wedge M \Rightarrow P$
 $B \wedge L \Rightarrow M$
 $A \wedge P \Rightarrow L$
 $A \wedge B \Rightarrow L$
 A
 B

Forward chaining example



$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

$$B \wedge L \Rightarrow M$$

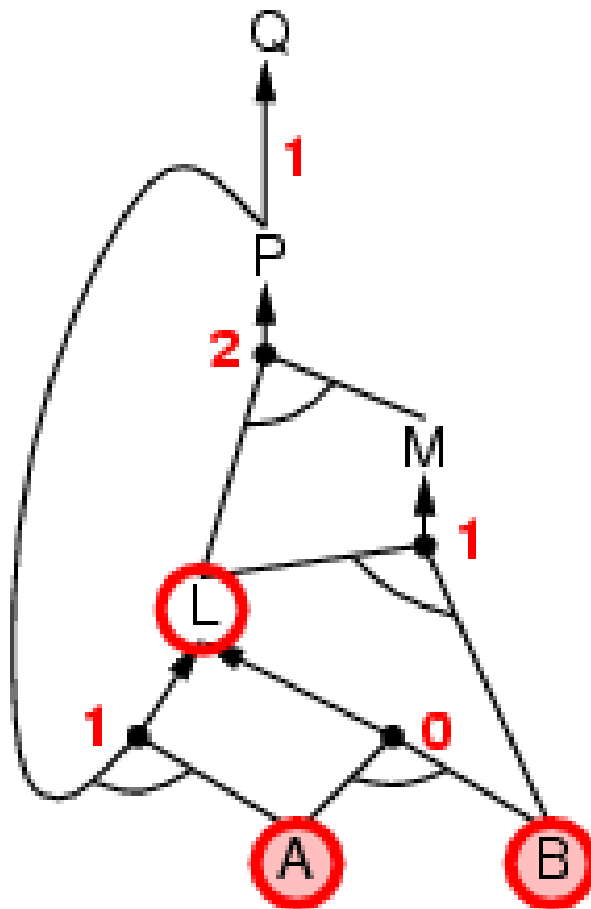
$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

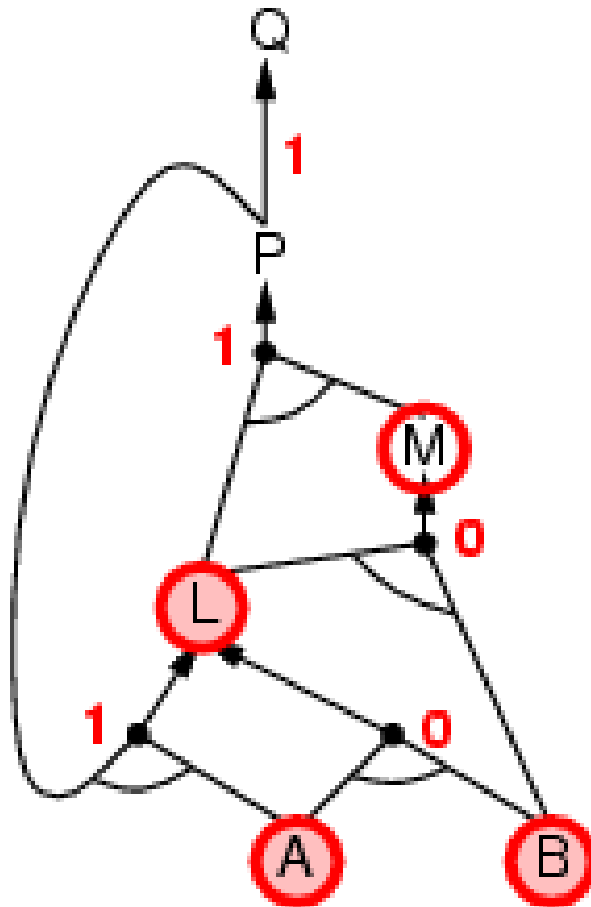
B

Forward chaining example



$P \Rightarrow Q$
 $L \wedge M \Rightarrow P$
 $B \wedge L \Rightarrow M$
 $A \wedge P \Rightarrow L$
 $A \wedge B \Rightarrow L$
 A
 B

Forward chaining example



$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

$$B \wedge L \Rightarrow M$$

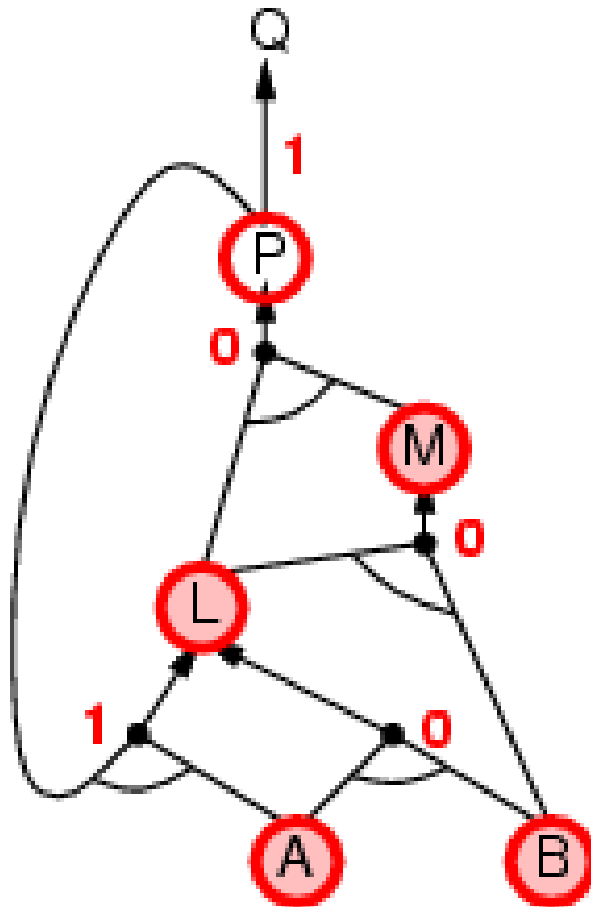
$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B

Forward chaining example



$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

$$B \wedge L \Rightarrow M$$

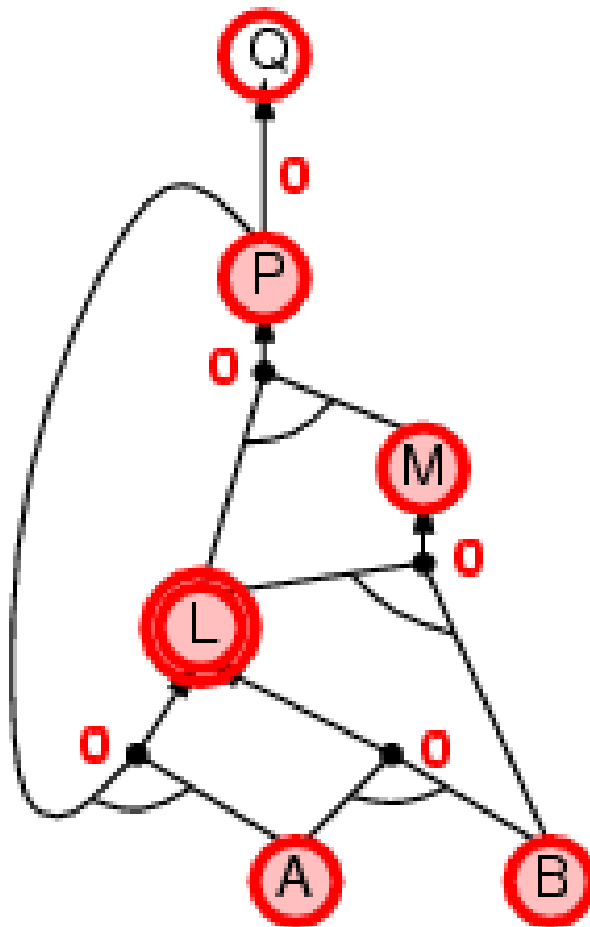
$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B

Forward chaining example



$P \Rightarrow Q$

$L \wedge M \Rightarrow P$

$B \wedge L \Rightarrow M$

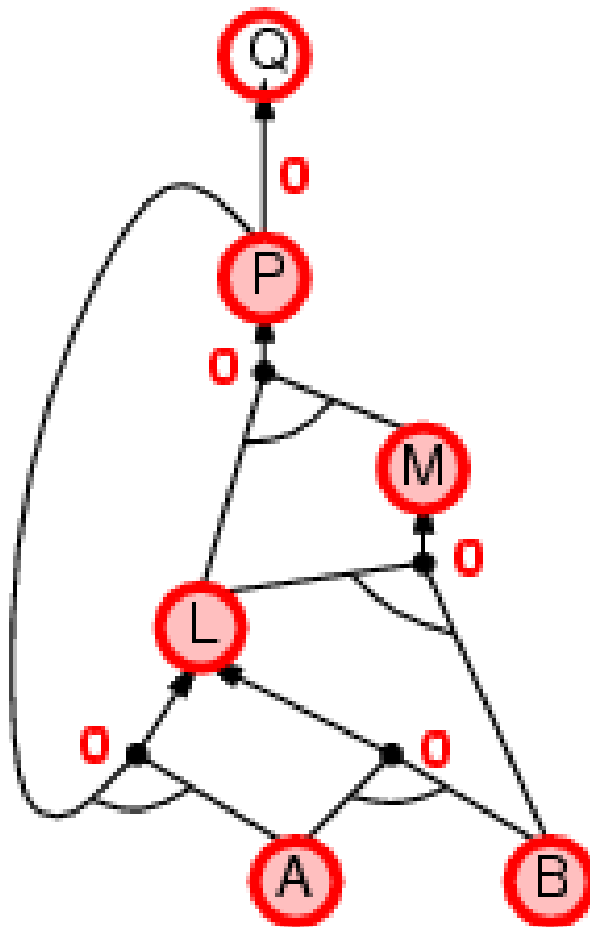
$A \wedge P \Rightarrow L$

$A \wedge B \Rightarrow L$

A

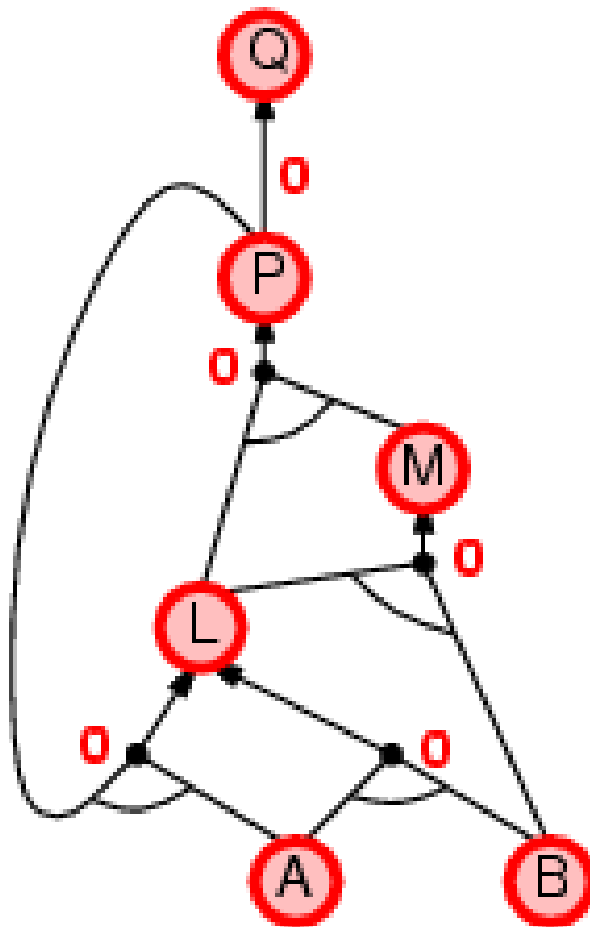
B

Forward chaining example



$P \Rightarrow Q$
 $L \wedge M \Rightarrow P$
 $B \wedge L \Rightarrow M$
 $A \wedge P \Rightarrow L$
 $A \wedge B \Rightarrow L$
 A
 B

Forward chaining example



$P \Rightarrow Q$
 $L \wedge M \Rightarrow P$
 $B \wedge L \Rightarrow M$
 $A \wedge P \Rightarrow L$
 $A \wedge B \Rightarrow L$
 A
 B

Proof of completeness

- FC derives every atomic sentence that is entailed by KB
 1. FC reaches a **fixed point** where no new atomic sentences are derived
 2. Consider the final state as a model m , assigning true/false to symbols
 3. Every clause in the original KB is true in m
$$a_1 \wedge \dots \wedge a_k \Rightarrow b$$
 4. Hence m is a model of KB
 5. If $KB \not\models q$, q is true in **every** model of KB , including m

Backward chaining

Idea: work backwards from the query q :

- to prove q by BC,

- check if q is known already, or

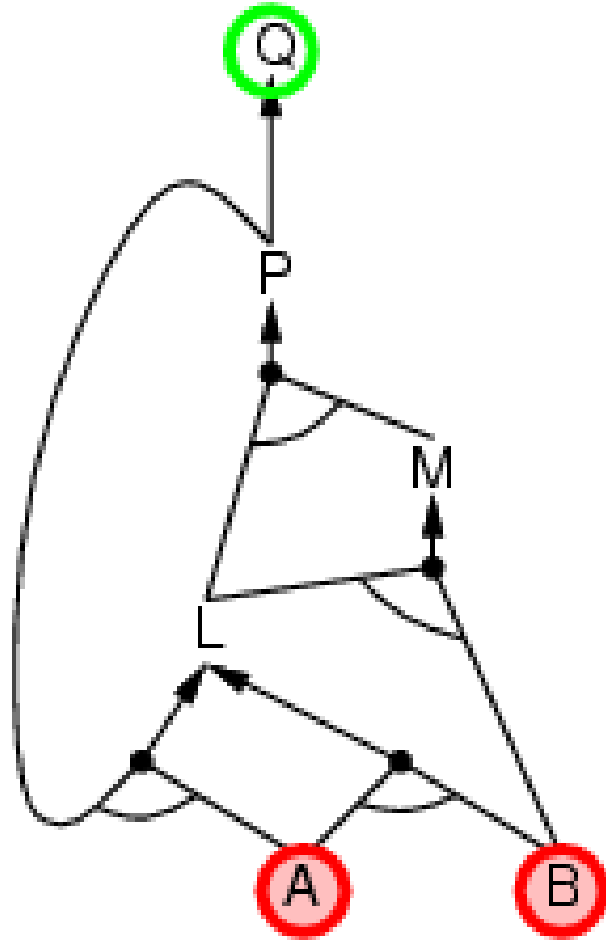
- prove by BC all premises of some rule concluding q

Avoid loops: check if new sub-goal is already on the goal stack

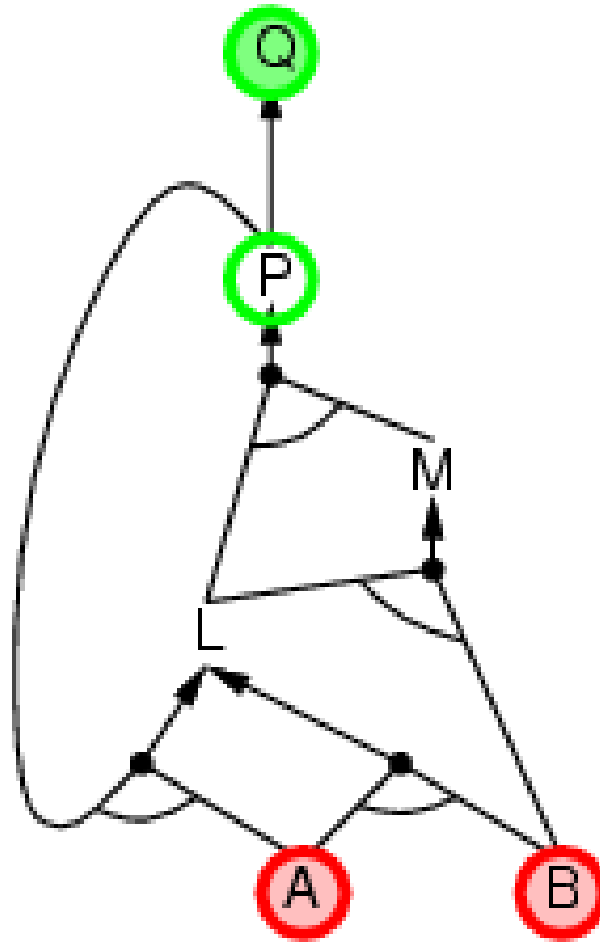
Avoid repeated work: check if new sub-goal

1. has already been proved true, or
2. has already failed

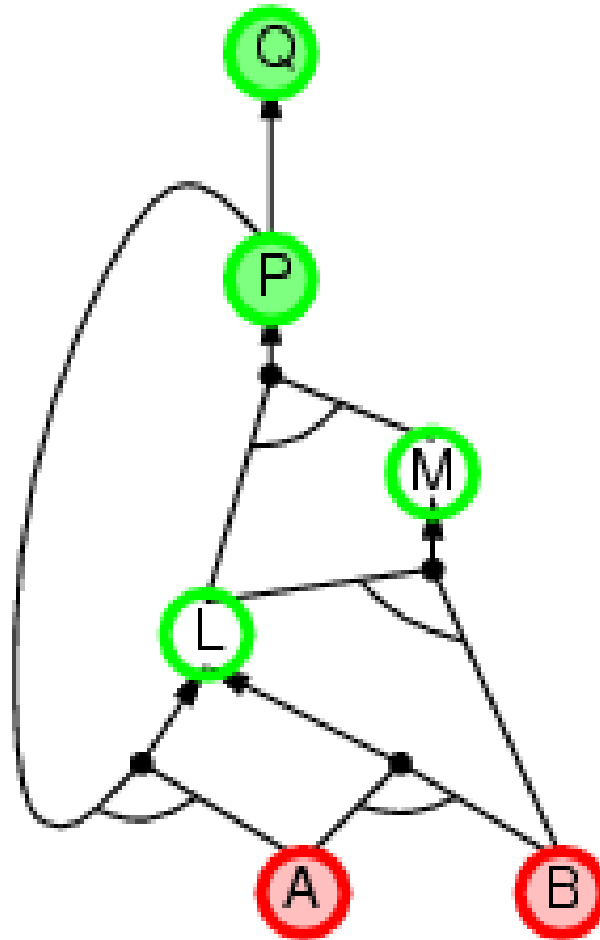
Backward chaining example



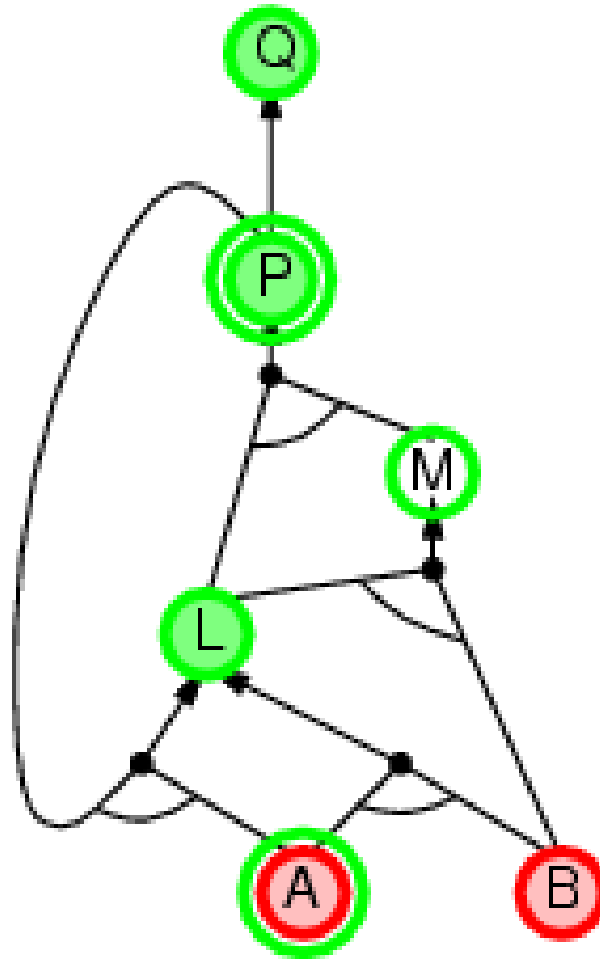
Backward chaining example



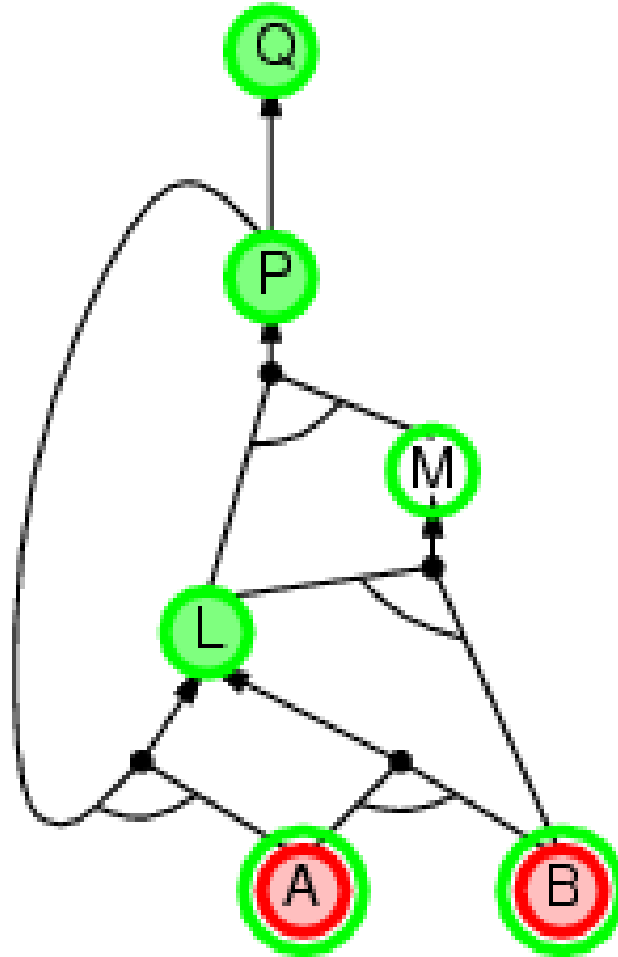
Backward chaining example



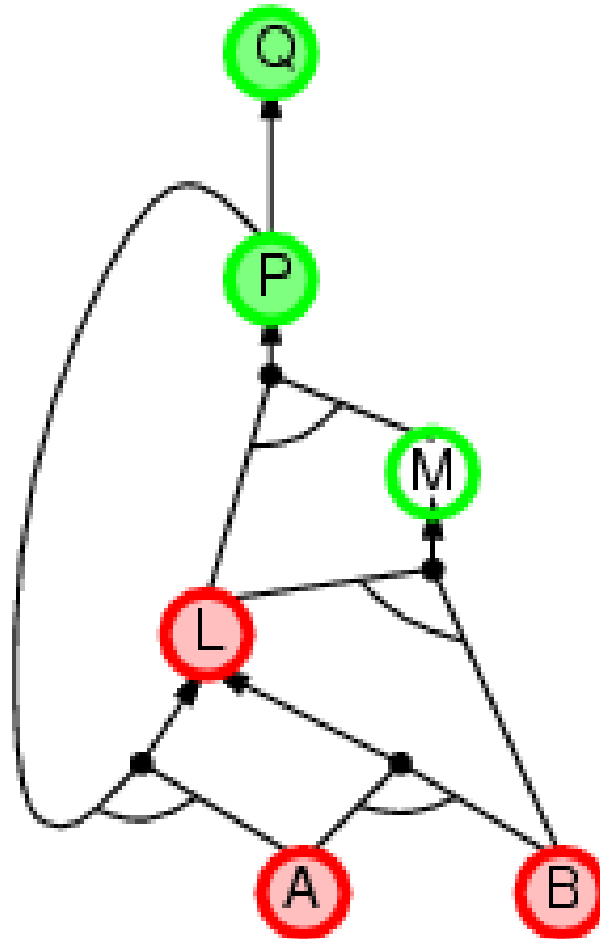
Backward chaining example



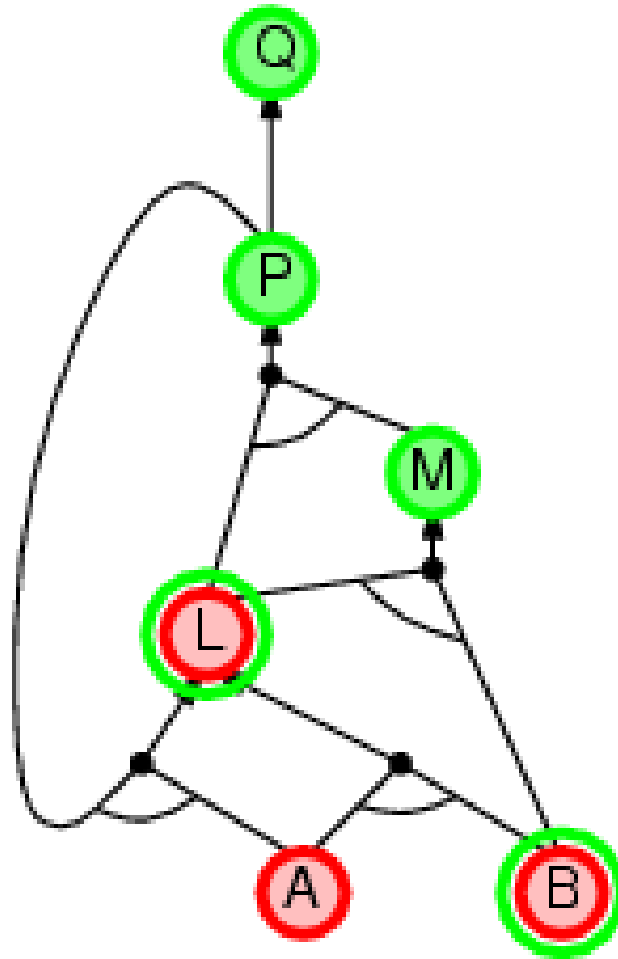
Backward chaining example



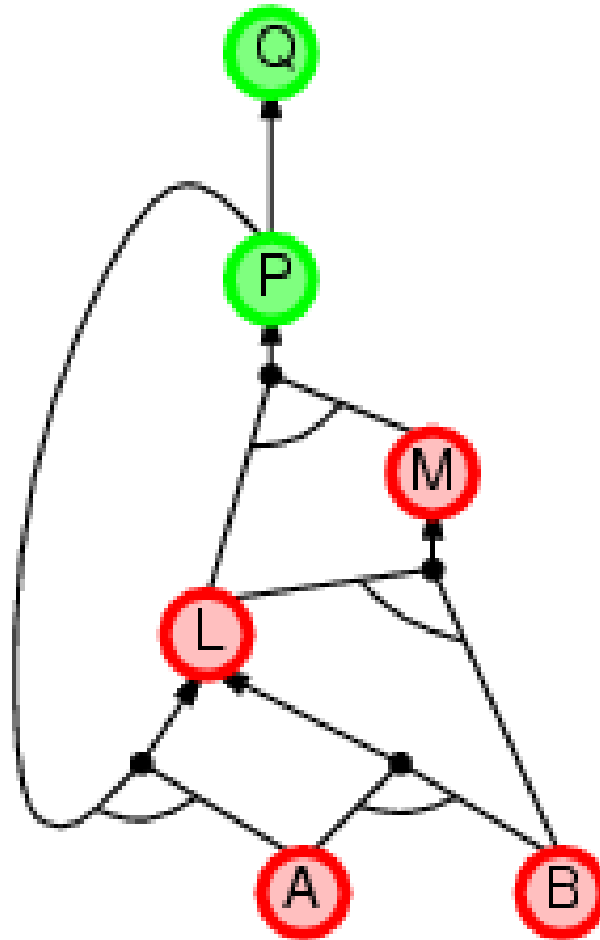
Backward chaining example



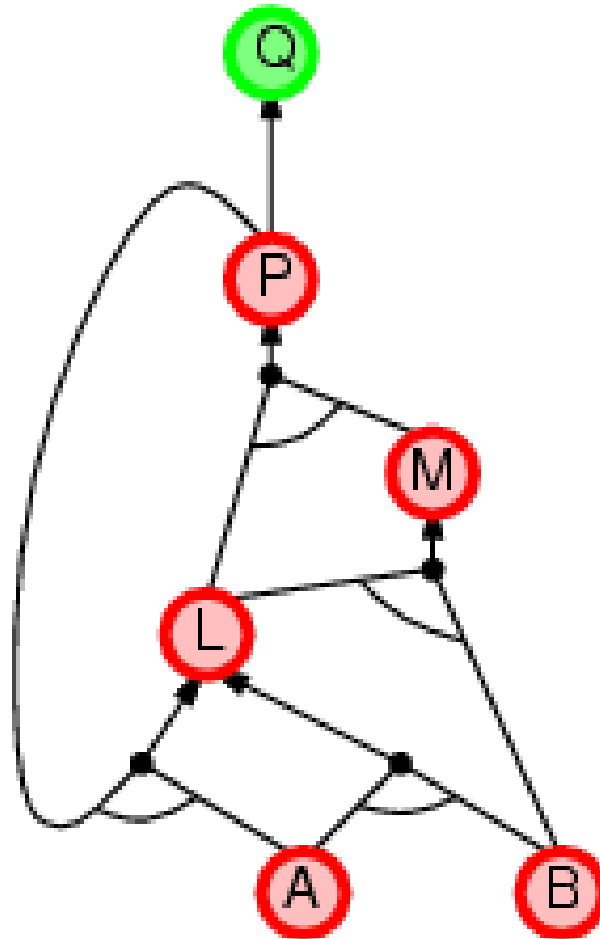
Backward chaining example



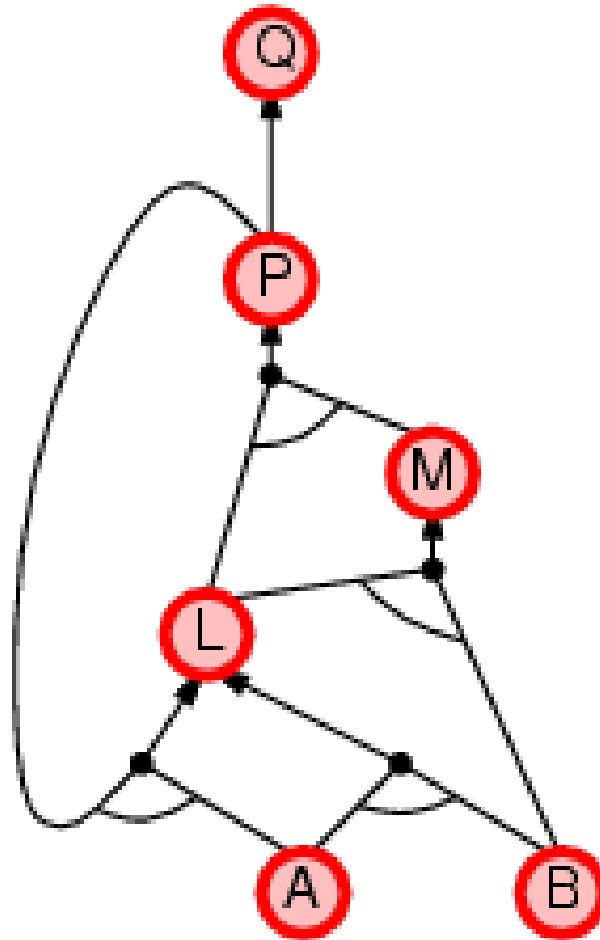
Backward chaining example



Backward chaining example



Backward chaining example



Forward vs. backward chaining

- FC is **data-driven**, automatic, unconscious processing,
 - e.g., object recognition, routine decisions
- May do lots of work that is irrelevant to the goal
- BC is **goal-driven**, appropriate for problem-solving,
 - e.g., How do I get into a PhD program?
- Complexity of BC can be **much less** than linear in size of KB

PL is Too Weak a Representational Language

- Consider the problem of representing the following information:
 - Every person is mortal. (S1)
 - Confucius is a person. (S2)
 - Confucius is mortal. (S3)
- S3 is clearly a logical consequence of S1 and S2. But how can these sentences be represented using PL so that we can infer the third sentence from the first two?
- We can use symbols P, Q, and R to denote the three propositions, but this leads us to nowhere because knowledge important to infer R from P and Q (i.e., relationship between being a human and mortality, and the membership relation between Confucius and human class) is not expressed in a way that can be used by inference rules

- Alternatively, we can use symbols for parts of each sentence
 - $P = \text{"person"}; M = \text{"mortal"}; C = \text{"Confucius"}$
 - The above 3 sentences can be roughly represented as:
 $S2: C \Rightarrow P; S1: P \Rightarrow M; S3: C \Rightarrow M.$
 - Then $S3$ is entailed by $S1$ and $S2$ by the chaining rule.
- Bad semantics
 - “Confucius” (and “person” and “mortal”) are not PL sentences (**not a declarative statement**) and cannot have a truth value.
 - What does $P \Rightarrow M$ mean?
- We need infinite distinct symbols X for individual persons, and infinite implications to connect these X with P (person) and M (mortal) because we need a unique symbol for each individual.

$\text{Person}_1 \Rightarrow P; \text{person}_1 \Rightarrow M;$
 $\text{Person}_2 \Rightarrow P; \text{person}_2 \Rightarrow M;$
 $\dots$
 $\text{Person}_n \Rightarrow P; \text{person}_n \Rightarrow M$

Weakness of PL

- Hard to identify "**individuals**." E.g., Mary, 3
 - Individuals cannot be PL sentences themselves.
- Difficult to directly and clearly talk about **properties** of individuals or **relations** between individuals (hard to connect individuals to class properties).
 - E.g., property of being a human implies property of being mortal
- Generalizations, patterns, regularities can't easily be represented.
 - All members of a class have this property
 - Some member of a class have this property
- A better representation is needed to capture the relationship (and distinction) between objects and classes, including properties belonging to classes and individuals
- **First-Order Logic (abbreviated FOL or FOPC) is expressive enough to concisely represent this kind of situation by separating classes and individuals**
 - Explicit representation of individuals and classes, x, Mary, 3, persons.
 - Adds relations, variables, and quantifiers, e.g.,
 - “*Every person is mortal*” Forall X: person(X) $\Rightarrow$ mortal(X)
 - “*There is a white alligator*” There exists some X: Alligator(X) $\wedge$ white(X)